university of groningen / faculty of science and engineering / artificial intelligence

The Stability Gap as a Trajectory Problem: A Bent Descent Path, Its Overshoot, and One Gate in Two Designs

Bachelor's Thesis

Arthur Ernst Henrik Reuss, s5582253, a.e.h.reuss@student.rug.nl,
Supervisor: Gido van de Ven

Abstract: Replay-based continual learning is usually judged by the accuracy it reaches *after* each task. Continual evaluation exposes what this view hides: the *stability gap*, a sharp but transient drop in past-task accuracy at the start of every new task. The gap persists even when the optimiser descends the exact joint loss, so at least part of its cause lies in the optimisation trajectory rather than the objective. This thesis explains the gap on two levels. The *intended trajectory*, the path exact gradient descent would take, bows out of the old task's basin before returning; and the followed path overshoots the intended one through everything that vanishes with the step size: discrete updates, momentum, and gradient noise. We separate the two levels by measurement rather than by argument. Rescaling the step size is a reparametrisation of the same gradient flow, so a ladder of learning rates run at matched flow time from one frozen checkpoint discretises one and the same intended trajectory, and whatever survives $\eta \to 0$ is that trajectory. On rotated MNIST the ladder converges, and not on zero: of the 23 pp drop at the standard step size, between two thirds and three quarters is discretisation overshoot, and a smooth arc of some four points remains that neither an exact replay gradient nor any further shortening of the step removes. Reaching that limit costs a hundred times the training, which sets the bar for a method: not lowering the gap, which a shorter step does with no method at all, but lowering it at the standard step size and in the standard number of steps. Written in a unified update rule, replay methods counter both levels with a single tool, a *gate* on the new-task gradient with the replay gradient restoring at full strength, and we build and compare the two pure gate designs: an adaptive λ-curriculum (scalar, feedback) and asymmetric Fisher-preconditioned replay (directional, feedforward). The curriculum earns its keep only under momentum, and there it cuts three quarters of vanilla ER's depth at vanilla accuracy, matches in one epoch what shortening the step reaches only in the limit, and carries to CIFAR-10 on a ResNet-18 at negligible overhead. The directional gate protects the past task at no measured plasticity cost but at far higher compute, bends the arc itself on the diagnostic full-buffer leg, fails late and abruptly at its strongest setting there, and is re-inflated by momentum. Which gate is preferable is set by the momentum regime.

Contents

1 Introduction

Deep neural networks learn powerful representations from static, independent, and identically distributed data, but a vulnerability appears as soon as training becomes sequential: the network overwrites previously acquired knowledge to accommodate new data. This phenomenon, catastrophic forgetting [18, 22], is a major hurdle in machine learning. Retraining from scratch solves the issue, but it is too expensive for systems that need to adapt continuously. Continual learning (CL) studies how to learn from a sequence of tasks without that cost, and its core difficulty is the stability–plasticity dilemma: the model must stay plastic enough to learn new distributions while staying stable enough to preserve old ones [19].

Replay, the family this thesis studies, stores a small buffer of past examples and trains on them alongside the new data. Replay methods reduce long-term forgetting well, but until recently they were evaluated only at task boundaries, which hides what happens within a task. De Lange et al. [4] evaluated the network after every update and identified the *stability gap*: a severe but transient collapse in past-task accuracy that occurs immediately when a new task begins, followed by a slow recovery. This matters in deployment: a system is queried while it trains, so a model caught mid-transition returns wrong answers on inputs its final checkpoint would handle correctly.

The discovery raised a natural question: would a better approximation of the joint loss over all tasks remove the gap? Hess et al. [12] showed that the gap persists even under incremental joint training, where the model has access to all data from all tasks at every step. A perfect objective alone does not remove transient forgetting. The problem therefore lies in *how* the optimisation proceeds, and this thesis locates it on two levels: the *intended trajectory*, the path exact gradient descent would take, already bows out of the old task's basin; and optimisers at a step size greater than zero overshoot the trajectory they intend to follow.

Two levels. Figure 1.1 visualises the transition as a switch from a learned task (Task A) to a new task (Task B), a conceptual reduction of a high-dimensional surface using the loss-landscape lens of Kao et al. [14]. *Level one* is geometric: steepest descent on the abruptly assembled joint loss (green curve) bows out of Task A's basin and returns from outside [14], so the Task-A loss rises above its final level and comes back, its recovery slow and lingering, the *hanging tail*. The excursion is a property of the path, not a necessary cost: Section 3.1 constructs a reference path along which the past-task loss rises monotonically to its joint-optimum level (the dashed chord), valid where its local curvature condition holds, so under that condition everything the accuracy curve does below its settled level and back is avoidable in principle. *Level two* is that optimisers overshoot the trajectory they intend to follow: at a step size greater than zero the large, unopposed boundary gradient [4] is applied in a few discrete jumps rather than traversed as a curve, and finite-buffer gradient noise perturbs every step [1]. Section 3 develops both levels.

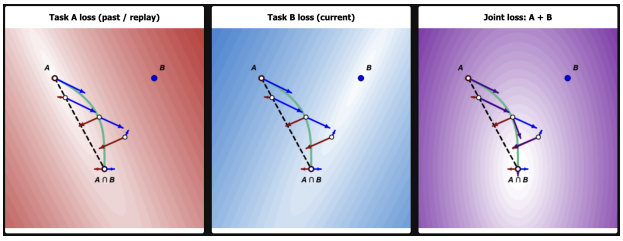


Figure 1.1: 2D optimisation landscape at a task transition. Single-task losses Task A (red) and Task B (blue) are minimised at A and B; the joint loss (purple) is minimised at $A \cap B$. The steepest-descent trajectory (green curve) illustrates the trajectory problem (Section 3.2), bowing outside A's basin into regions of higher loss. In contrast, the dashed black chord sketches the monotone reference path of Section 3.1, which stays inside the basin. Overshoot drivers (Section 3.2) further push the iterate off these paths. An interactive version, in which landscapes, task minima, and both gates of Section 3.3 can be manipulated, is available at `https://arthurreuss.github.io/ValleyWalk/loss_landscape_visualization.html`.

One tool: gate the new-task gradient. Sequential replay methods share a single update, a weighted combination of the current-task and replay gradients [24]. Section 3.3 generalises this: because only the ratio of the two weights sets the update direction, any reweighting of this update can be rewritten with the replay gradient restoring at full strength and a single *gate* on the new-task gradient, $d = G\, g_{\text{cur}} + g_{\text{rep}}$. The gate's *domain* can be scalar (attenuate the whole push) or directional (attenuate per direction), and its *control* feedback (react to a live damage signal) or feedforward (act through a schedule or a curvature model). This thesis builds and compares the two pure designs. The first is the **λ-curriculum**, a scalar gate: instead of switching abruptly to the joint loss, it ramps the new-task loss weight $\lambda(t)$ from 0 to 1, so the objective's minimiser moves continuously from θ^*_A to θ^*_{joint} and the model tracks a moving valley floor. Its adaptive variant sets λ from the ratio of the replay and new-task gradient norms, a live damage signal, which makes it a scalar *feedback* gate. The second is **asymmetric preconditioned experience replay** (asymmetric PER), a directional *feedforward* gate: it filters the new-task gradient through the curvature of the replay loss, passing the push at full gain where the past task is flat and braking it where it is steep, leaving the restoring gradient untouched.

2 Background and Related Work

2.1 Continual Learning and Experience Replay

Methods against catastrophic forgetting fall into a few families: replay, parameter and functional regularisation, optimisation-based methods, context-dependent processing, and template-based classification [27]. This thesis focuses on replay, specifically its simplest formulation: Experience Replay (ER) [3]. At each step, ER draws a replay mini-batch from a buffer of stored past examples, combines it with the current-task mini-batch, and takes one optimiser step on the summed loss.

A standard approach to managing this episodic memory is reservoir sampling [28], which ensures that every sample seen so far occupies a buffer slot with equal probability. Under this setup, the buffer remains empty during the first task, meaning ER reduces to plain SGD; consequently, performance gaps and forgetting metrics are evaluated from the second task onward. Despite this simplicity, ER remains highly competitive: a small episodic memory often outperforms purpose-built methods and scales to longer sequences [3]. Furthermore, because the ER update consists of exactly two ingredients, it provides a remarkably clean substrate for isolating and studying interventions at task boundaries.

However, a finite buffer is inherently an imperfect estimator of the past-task distribution. Aljundi et al. [1] frame buffer selection as approximating the feasibility region of the full past data, with reservoir sampling serving as one specific policy within that framework. Consequently, the replay gradient derived from a small buffer will differ in both direction and magnitude from the true gradient of all past data.

2.2 The Stability Gap

De Lange et al. [4] identified the stability gap using continual evaluation, measuring past-task accuracy after every update rather than only at task boundaries. In this view a new task begins with substantial but transient forgetting that the model subsequently recovers from, and they quantify it with worst-case metrics, mainly the minimum past-task accuracy reached during training, rather than with end-of-task averages alone. The effect is not confined to one method: they observe it across experience replay, constraint-based replay, knowledge distillation, and parameter regularisation. It also persists well beyond the standard benchmarks, appearing in joint incremental learning of homogeneous tasks [13] and in continual pre-training of large language models [8], and several mitigations have been proposed [10, 25].

Two findings in this literature anchor the account of Section 3. First, De Lange et al. [4] trace the initial drop to a gradient imbalance at the boundary: the new-task gradient dominates the first updates, so the optimiser trades stability for plasticity precisely when the past task is most exposed. Second, the gap is a property of the trajectory rather than the objective. Hess et al. [12] show it persists under training on the exact joint loss with full access to all past data, and Kao et al. [14] demonstrate the geometric mechanism analytically: steepest descent from an old-task minimum leaves the old basin even with perfect joint gradients, because the steepest direction and the direction to the joint minimum disagree. A perfect objective alone therefore fails to prevent the gap, and the descent geometry itself must be addressed. The optimiser matters as well: comparing optimisers in a similar setting, Obis [21] find the depth and duration of the drop to depend substantially on which one is used, an early indication that the effective step size is a lever on the gap.

2.3 One Update, Many Methods

Beyond plain ER, several methods constrain the update at the task boundary using information about past tasks. GEM stores past examples and, at every step, recomputes each past task's gradient on its stored data, then projects the new-task gradient onto the nearest direction that leaves every stored loss unchanged or lower [16]. A-GEM replaces the per-task constraints with a single averaged past-task gradient, reducing the projection to one cheap operation [2]. EWC adds a quadratic penalty around the past optimum weighted by a curvature estimate [15], and natural-gradient approaches precondition the update with past-task curvature so that steps avoid directions the old task depends on [14].

These methods are not all of one kind: EWC and the natural-gradient family act through a penalty or a preconditioner rather than through replayed data, and they do not fit the form below. The replay-based methods, however, share a single update, as Sodhani et al. [24] show. Every step combines two gradients, the current task's and the replay buffer's, under two scalar weights,

$$\theta \leftarrow \theta - \eta \left(\alpha_1 \nabla L_{\text{current}} + \alpha_2 \nabla L_{\text{replay}}\right), \qquad (2.1)$$

so a replay method reduces to a rule for choosing α_1 and α_2. Plain ER fixes both to one. A-GEM holds $\alpha_1 = 1$ and raises the replay weight α_2 only when the new-task and stored gradients conflict, by just enough to cancel the interference, so its projection is equivalent to re-weighting the two losses; GEM does the same per stored task, through a quadratic program whose multi-constraint solution no longer reduces to two scalar weights. MEGA [9] likewise holds $\alpha_1 = 1$ but sets the replay weight from a loss ratio,

$\alpha_2 = L_{\text{replay}}/L_{\text{current}}$, leaning on memory once the new task is already fit.

The same reading applies to the λ-curriculum of this thesis. It fixes $\alpha_2 \equiv 1$ and turns the other knob, ramping the current-task weight $\alpha_1 = \lambda(t)$ up from zero. It belongs to the same family as A-GEM and MEGA; what distinguishes it is which weight it moves and what signal drives the weight. The adaptive variant (Section 3.3) is the mirror image of MEGA: MEGA raises the *replay* weight from a *loss* ratio to recruit memory once the new task is learned, while the adaptive curriculum holds the *new task* back with a *gradient-norm* ratio, letting it in only as the replay gradient reasserts itself.

2.4 Low-Loss Paths and Mode Connectivity

Mode connectivity gives empirical backing to the *monotone reference path* introduced in Section 3: a route from θ^*_A to θ^*_{joint} along which the past-task loss only ever rises toward its final value requires the two solutions to be joined by a corridor of low loss, and this is what the literature reports. Curved paths of near-constant low loss join independently trained minima [5, 7], straight ones join networks that share early training [6], and, for continual learning specifically, the sequentially trained and multitask solutions often lie in the same basin [20]. Where such a route exists, the stability gap reflects the optimiser wandering off it, and an intervention succeeds by keeping it there.

3 The Stability Gap as a Trajectory Problem

This section reframes the stability gap as a two-level trajectory problem. Learning a new task incurs a permanent stability-plasticity cost; the gap is the transient excess above it. This excess has two sources: a geometric bow in the *intended trajectory*, the path that exact gradient descent would follow, and the overshoot that discrete, finite-step training adds on top of it. We formalise this split by constructing a monotone reference path. We then show that gating the new-task gradient counters these deviations, though no single pure gate design neutralises every driver at once. Instead, how the update is gated dictates which source of error is suppressed and which remains exposed. Two contrasting gates instantiate this: an adaptive scalar curriculum and an asymmetric directional filter. One register note applies throughout: on the quadratic model of this section the claims are exact; carried to deep networks they are hypotheses, which Section 6 tests and Section 7 weighs.

3.1 The Reference Path and the Tracking Bound

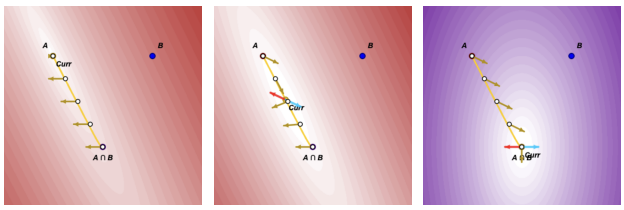
Learning a second task inevitably moves the model. In a quadratic approximation with task losses L_A and L_B and Hessians H_A and H_B, the displacement $d = \theta^*_{\text{joint}} - \theta^*_A$ from the old optimum to the joint minimum permanently raises the replay loss by roughly $\frac{1}{2}\, d^\top H_A\, d$. Every method pays this price; it is distinct from the stability gap.

Yet this rise can be strictly monotone. Consider the curriculum objective $L_\lambda = L_{\text{replay}} + \lambda L_{\text{current}}$; at $\lambda = 1$ it is the joint loss with the two tasks weighted equally, the convention used throughout. As λ grows from 0 to 1, the minimiser traces a *reference path* $\Theta^*(\lambda)$ from θ^*_A to θ^*_{joint}. Under local positive-definiteness, an implicit-differentiation argument along the minimiser branch (Appendix D) guarantees that the replay loss climbs directly to its final level with no excursion above it:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0 \qquad \text{for all } \lambda \in [0,1]. \tag{3.1}$$

This guarantee is local: it holds only while the combined Hessian $H_A + \lambda H_B$ stays positive-definite along the tracked branch of minima. The condition is automatic near θ^*_A and persists until the new task's negative curvature overpowers the old task's; on a deep non-convex backbone it therefore serves as motivation rather than a guarantee.

Figure 3.1 shows this: as λ grows, the loss contours deform continuously from the old-task basin to the joint basin and the minimiser slides along the connected valley floor. Where the condition holds, the replay loss climbs directly to its final level, and any transient excess *above* it, the stability gap, is a preventable deviation from the reference path.



$\lambda = 0$ $\lambda = 0.025$ $\lambda = 1$

Figure 3.1: Evolution of the curriculum loss $L_\lambda = A + \lambda B$. As λ increases, the contours morph continuously from the old-task basin around A to the joint basin around $A \cap B$, and the current minimiser (*Curr*) slides along the moving valley floor, forming the reference path $\Theta^*(\lambda)$. At the minimiser the replay (red) and current-task (blue) contributions cancel by the first-order condition $\nabla L_{\text{replay}} + \lambda \nabla L_{\text{current}} = 0$. Frames from the interactive visualisation of Figure 1.1.

A real iterate cannot sit on this path exactly, but it can stay close to it. Introducing the new task gradually over an N-step ramp ($\lambda \approx 1/N$) keeps the iterate

tethered to the moving valley floor $\Theta^*(\lambda(t))$ rather than letting it leap across the landscape; a gradient-flow analysis bounds the resulting lag by $O(1/N)$ (Appendix D). Because the monotone path fixes the endpoint at the permanent level, this caps the highest transient replay loss of the whole transition,

$$\max_t L_{\text{replay}}(\theta(t)) \;\le\; L_{\text{replay}}(\theta^*_{\text{joint}}) + O\left(\frac{1}{N}\right), \quad (3.2)$$

the first term the permanent price, the second a transient that shrinks with ramp length. A slow ramp is precisely what the λ-curriculum of this thesis implements.

3.2 The Two-Level Gap: Bowing and Overshoot

Real trajectories deviate from the reference path on two distinct levels, and the step size is what separates them.

Level One (the intended trajectory and its bow): The *intended trajectory* is the path exact gradient descent would follow on the replay objective: gradient flow, the limit of vanishing step size, computed with exact gradients. Even this idealised path bows. The steepest-descent direction of the abruptly assembled joint loss misaligns with the direction toward the joint minimum whenever the loss curves more sharply in some directions than in others, so the path leaves the old task's basin before returning [14]. The gradient imbalance at the task boundary belongs to this level. At θ^*_A the replay gradient vanishes because the iterate sits at the replay optimum, while the new-task gradient is large [4]; the intended trajectory therefore sets off along the new-task direction, and the imbalance shapes where the bow bends. It is a property of the vector field the descent follows, not of how that field is discretised. This deterministic out-and-back arc forms the structural core of the gap (Figure 1.1, green curve).

Level Two (Overshoot of the followed path): The *followed path* is the trajectory the optimiser actually traces. It deviates from the intended trajectory through two fundamental sources of error that only manifest at a finite step size η: discrete updates cut straight chords across the curving landscape (geometric errors that momentum further accumulates), and sampling noise corrupts the buffer-estimated replay gradient (Section 2.1). A single parameter therefore governs this entire level, acting through two distinct drivers. The *acute driver* is the finite step size meeting the massive, unopposed boundary field: the very first update commits to the full new-task gradient over a displacement of $\eta \, \|g_{\text{cur}}\|$, causing the iterate to jump along the new-task direction before the growing replay gradient can pull it back. Within a handful of steps, it sits much further out than the intended trajectory ever goes, producing the sharp first-step drop. The *chronic driver* is the ongoing sampling noise, which corrupts the restoring gradient at every step, slows the recovery tail, and can trigger sudden drops long after the boundary has been crossed.

We hypothesise that these two levels shape the two dimensions of the stability gap: depth records the overshoot, whether from the acute boundary spike or a late, noise-driven drop, while the hanging tail consists of the Level-One arc combined with the slow, noisy recovery the chronic driver leaves behind. This mapping forms our falsifiable core, formalised as RQ1 in Section 4: interventions on the step, slowing it or denoising it, should mitigate the drop but leave the arc intact; placing the optimiser onto the reference path should instead remove the arc; and only combining both should close the gap completely, while the permanent cost of Section 3.1 remains.

3.3 Two Gates: A Division of Labour

Equation (2.1) shows that replay methods differ mainly in how they weight the replay and current-task gradients. Only the ratio of the two weights sets the update *direction*; their common scale sets the length of the step. In gradient flow the scale can be absorbed into the learning rate exactly, so any such method can be rewritten with the replay weight normalised to one. At a step size greater than zero, however, the absorbed scale *is* the effective learning rate, and Section 3.2 identifies the step size as the acute Level-Two driver; the choice of scale is therefore not harmless. We keep the normalisation but fix the learning rate across all methods and conditions, so every comparison runs at a matched step size. Furthermore, we measure the step size's own contribution in a dedicated block, the step-size ladder of Section 5, which drives η towards zero at matched flow time and leaves the objective, the buffer, and the update direction untouched [21].

The replay gradient is the only restoring force pulling the iterate back to the reference path, so a protective method must attenuate the new-task push, the source of the bow and overshoot, while leaving the restoring gradient intact. Generalising the current-task weight to an operator G yields a one-sided *gate*:

$$d \;=\; G\, g_{\text{cur}} \;+\; g_{\text{rep}}. \quad (3.3)$$

The curriculum of Section 3.1 realises this gate along one axis, *time*: it introduces the new task slowly, so the push is never large enough at once to knock the iterate off the reference path. A complementary axis is *direction*. At any single step only part of the push is harmful, because the past task is flat in most directions and steeply curved in a few, and only motion along the steep directions raises the replay loss. Preconditioning the push by past-task curvature exploits this, passing it at full gain where the past task is flat and shrinking it where the past task is steep, so retention costs little plasticity. This natural-gradient idea

underlies Natural Continual Learning [14] and, in a replay setting, Online Curvature-Aware Replay [26]. Both, however, precondition the *entire* update, so the same metric that brakes the harmful push also brakes the restoring replay gradient, protecting the past task only by slowing the return to it. Asymmetric PER keeps the gate one-sided: it filters the new-task gradient alone and leaves the restoring gradient at full strength. We include the symmetric preconditioner, which damps both, as a control (Appendix C.5).

This thesis explores two specific gating mechanisms to implement this attenuation, alongside a fixed linear ramp that serves as an ablation:

Adaptive λ-curriculum: A scheduling parameter $\lambda(t)$ sets the new-task weight from the ratio of replay to new-task gradient norms, a live damage signal that fires whichever direction harm arrives from. By scaling the loss itself, it deforms the objective into the homotopy of Section 3.1, so the optimiser's natural descent follows the reference path.

Asymmetric PER: The gate $G = \delta(F_{\text{rep}} + \delta I)^{-1}$ is a filter set by the replay Fisher F_{rep}, giving the update

$$d \;=\; \delta\,(F_{\text{rep}} + \delta I)^{-1} g_{\text{cur}} \;+\; g_{\text{rep}}, \qquad (3.4)$$

with the damping δ its single knob: as $\delta \to 0$ the push concentrates in the directions the past task is flat in. The filter is rebuilt each step from the current parameters and past-task curvature.

These distinct mechanisms yield specific hypotheses regarding how each gate navigates the stability-plasticity dilemma, formalised as research questions in Section 4. While both gates aim to steer the intended trajectory back toward the monotone reference path, they tackle the Level Two deviations from fundamentally different angles. Because the curriculum holds back the entire new-task push at the source, it successfully mitigates the acute boundary overshoot, but it does so at a direct plasticity cost. Furthermore, it leaves chronic sampling noise exposed, which can lead to late, noise-driven drops. Consequently, it requires a separate accelerator to repay the plasticity debt (RQ2). Conversely, the asymmetric filter isolates the harmful push by direction, theoretically managing the noise and protecting the past task with a minimal plasticity penalty (RQ3). However, this step-level filtering leaves the method prone to acute overshoot at the start. This theoretical elegance also trades off against computational cost: while the curriculum requires only a cheap gradient-norm ratio per step, asymmetric PER must solve a linear system in the replay Fisher via conjugate gradients (Appendix A.5).

Crucially, we hypothesise that optimiser momentum will expose a failure axis that discriminates between the two gates based on *where* they attenuate the gradient. Because the curriculum attenuates at the *source* ($\lambda \approx 0$ early in training), the new-task gradient is structurally flattened at the loss level. With no large gradient to accumulate, momentum can safely compose with the gate. Here, momentum solves both of the curriculum's weaknesses simultaneously: it provides acceleration to repay the plasticity debt, and it acts as a smoothing mechanism to average out the chronic gradient noise, all without re-inflating the acute overshoot. In contrast, asymmetric PER attenuates at the *step*. The raw new-task gradient remains large, and the filter shrinks it step-by-step. Because momentum accumulates past velocities, it ultimately rebuilds the harmful displacement that the per-step filter attempts to remove. For the directional gate, momentum is therefore expected to restore plasticity while actively worsening the acute overshoot, undermining the initial protection and deepening the gap.

This division of labour illustrates that mitigating the stability gap is about understanding the specific mechanisms at play. These two approaches reveal how their specific gating mechanisms interact with the momentum regime.

4 Research Questions

The two-level account and tracking bound of Section 3 make falsifiable predictions about the stability gap and the behaviour of distinct gating mechanisms. To evaluate these predictions, our experiments are organised around four core questions. The first three are evaluated in a controlled setting (rotated MNIST with a small MLP) to probe the gap's dynamics step-by-step, while the fourth tests the generalisability of these findings under increased complexity.

RQ1: The account. A uniform rescaling of the learning rate is a time reparametrisation of the same gradient flow, so driving $\eta \to 0$ at matched flow time follows the intended trajectory ever more faithfully without changing it. **Does the stability gap separate under that limit into a deterministic arc that survives it, belonging to the intended trajectory, and an overshoot that exists only at finite step size?**

RQ2: The adaptive scalar gate. Taking the limit is itself a repair, but a costly one: a smaller step removes the overshoot and bills for it in training length, the epoch count scaling as $1/\eta$ at matched flow time. **Can an adaptive scalar feedback gate (the adaptive λ-curriculum) fix the trajectory without paying that bill, and how does the answer depend on optimiser momentum and on the plasticity the gate withholds?**

RQ3: The directional feedforward gate. To what extent does a directional feedforward gate (asymmetric PER) reduce the stability

gap, and how does accumulated momentum compromise this gating mechanism?

RQ4: Generalisation. Do the mechanisms underlying the stability gap, and the effectiveness of the proposed gates, generalise to deeper network architectures and more severe task shifts?

5 Methods

This section describes the empirical testbeds (Section 5.1), the metrics that summarise each run (Section 5.1.2), and the experimental blocks (Section 5.2) that answer the research questions of Section 4. All code, experiment configurations, and analysis scripts are available at `https://github.com/Arthurreuss/ValleyWalk`.

5.1 Experimental Setup

5.1.1 Testbeds

Two testbeds of different scale carry the study. A small, fast one (rotated MNIST on a two-layer MLP) runs the dense mechanistic sweeps: the step-size ladder, the curriculum sweep, the asymmetric-PER damping sweep, and the momentum cross. A larger one (domain-shift CIFAR-10 on a CIFAR-adapted ResNet-18) tests whether the findings survive a deeper backbone and a stronger task shift, the regime where the local condition behind the bound of Section 3.1 weakens. Both hold the label space and task structure fixed and shift only the input distribution, and both use two tasks in their main blocks, so there is exactly one transition and one instance of the stability gap; a five-task rot-MNIST sequence and a three-task CIFAR sequence serve as multi-boundary checks.

On rot-MNIST, T_0 presents the digits upright and T_1 rotates them by 90°, each task using the standard 60,000 training and 10,000 test images; the 90° domain shift follows De Lange et al. [4], which orients the pair as $-90° \rightarrow 0°$. On CIFAR-10, T_0 is clean and T_1 applies Gaussian-noise corruption at severity 3 [11]; the ten classes are shared, so only the input distribution shifts. A three-task variant ([none, Gaussian, shot]) and two alternative corruptions appear in the generalisation block. The rot-MNIST MLP is cheap enough to sweep across many conditions and seeds and is where the local condition of Section 3.1 is most plausible; the ResNet-18 is the deeper, more realistic counterpart. rot-MNIST runs online (one epoch per task); the ResNet needs several passes to converge before the transition, so the CIFAR block uses ten epochs per task. Optimiser, learning rate, and batch size are common across benchmarks; no method retunes the learning rate, so every comparison between methods runs at a matched step size (Section 3.3). The step size is held fixed in that role and manipulated in one block only, the ladder of Section 5.2.1, which holds the objective, the buffer, and the update rule fixed and varies η alone. Full backbone and protocol specifications are in Appendix A.2 (Tables A.1–A.3).

Throughout training every task's test accuracy is evaluated every 10 steps; at each transition the cadence rises to every step, so the dip is recorded at full resolution (the whole online epoch on rot-MNIST, a 250-step window on CIFAR). A stability-gap tracker snapshots each past task's pre-switch accuracy and records the resulting (step, accuracy) series; how the per-step figures draw the sparsely sampled pre-boundary segment is a plotting convention with no effect on any reported number, stated in Appendix A.2. On the vanilla-ER reference runs only, a gradient tracker additionally records the fidelity of the replay-buffer gradient against the exact past-task gradient, at the same cadence; each such step costs a full pass over the past-task training set, which is why it is not run elsewhere, the ladder's long rungs least of all.

5.1.2 Metrics

Each metric is reported as the seed mean ± sample standard deviation; complete definitions are in Appendix B. The gap definition of Section 3, the transient excess below the level a task settles at, gives two primary metrics. With a_j^{pre} the pre-switch baseline on past task j and $a_j(t)$ its accuracy at step t, `gap_depth` is the maximum instantaneous excursion $\max_j(a_j^{\mathrm{pre}} - \min_t a_j(t))$, which Section 3.2 predicts to be overshoot-dominated. The second, `gap_area_end`, integrates the shortfall below the settled level: with reference $b_j = \min(a_j^{\mathrm{pre}}, \bar{a}_j)$, where $\bar{a}_j$ is the accuracy the task eventually reaches, it sums $\max(0, b_j - a_j(t))$ over the new task's training steps and across past tasks. Anchoring to the recovered level implements the split of Section 3.1: a permanent step down to a lower plateau contributes to the standard forgetting metrics and near-zero to the area, while a true dip-and-recover registers in full. One consequence, revisited when reading the tables, is that two methods settling at different levels are measured against different lines, so a better-recovering method can carry the *larger* area; a positive area marks transient disruption, not net cost, and we read it with final accuracy. The arc of Section 3.2 and the slow part of the noisy recovery live here; sharp noise-driven drops register in depth instead. A third metric, `recovery_steps`, and the gradient-fidelity diagnostics are defined in Appendix B.

One clarification is needed for the ladder of Section 5.2.1, the only block in which η itself varies. `gap_depth` is a maximum over accuracies and is therefore η-invariant by construction. `gap_area_end` is not:

it is a trapezoidal integral of the drop curve against the step index, weighted by the spacing between records, with units accuracy·steps. Stretching a rung by $c = 0.1/\eta$ stretches its step axis by the same factor, so an identically shaped excursion reads c times larger. Across the ladder the area is therefore taken in flow time, $A_\tau = \eta\, A_{\text{steps}}$, the integral of the same curve against $\tau = \sum_t \eta$; within a rung the two differ by a constant, and every block at fixed η reports the step-axis value as before.

To place the gap in the standard frame we report the continual-evaluation metrics of De Lange et al. [4] from the task-boundary accuracy matrix: average final accuracy (ACC), which checks that a gate lowers the gap at preserved accuracy; worst-case retention (min-ACC, WC-ACC); and short-horizon plasticity (WP_{100}). FORG and the windowed WF_w/WP_w family are logged for completeness. Conditions are compared with the paired Wilcoxon signed-rank test on seed-paired differences, preferred over the t-test because five paired seeds cannot support a normality assumption on bounded metrics: the gap metrics floor at zero, where the strongest conditions cluster, and accuracy sits near its ceiling, so the seed differences are skewed rather than Gaussian. With five paired seeds the smallest two-sided p-value is 0.0625, reached when all five differences share a sign; we read $p = 0.0625$ as complete directional agreement and let effect sizes separate large results from marginal ones.

5.2 Experiments

Each block maps to one or more research questions and tests the corresponding predictions of Section 3. Unless otherwise noted, all conditions use five seeds and follow the standard protocol. Vanilla ER at the standard operating point (1k reservoir, $\eta = 0.1$, no schedule, and no filter) serves as the common reference baseline for every rot-MNIST block and is run at both momentum settings.

5.2.1 The Step-Size Ladder (RQ1)

To address RQ1, the step-size ladder isolates the effects of discretisation from the intended optimisation trajectory. A uniform rescaling of η does not change the underlying vector field. Because forward Euler at step η integrates the same flow $\dot{\theta} = -g(\theta)$ on a finer grid, a smaller step size follows the intended path more accurately. Therefore, as η decreases, any reduction in the performance gap represents discretisation error. Whatever error survives at the limit belongs to the continuous path the optimiser was trying to follow, which we define as the arc.

To ensure the rungs of the ladder represent a true limit rather than unrelated runs, we enforce four constraints. First, we match the total flow time: the epoch count scales as $1/\eta$, meaning a rung at $\eta = 0.1/c$ runs for c epochs. This guarantees every cell covers the same flow time $\tau = 23.5$ over the new task, preventing artificial performance gaps caused simply by under-training. Second, we use a frozen θ_0^*: every cell warm-starts from a shared task-0 checkpoint (Appendix A.4) rather than training the initial task at its own step size. This prevents smaller step sizes from settling into differently sharp minima before the task switch. Third, we maintain a matched record grid, scaling the evaluation cadence by c so every cell records the exact same number of samples spaced equally in flow time. Finally, the evaluation cadence is counted directly from the task boundary to ensure we capture the initial post-switch spike perfectly.

Table 5.1 details the ladder rungs: $\eta = 0.1$, $\eta \approx 0.033$, $\eta = 0.01$, and $\eta = 0.001$. The first three establish the $\mu = 0$ limit, while the fourth accommodates the momentum leg. Each rung is evaluated on two buffer arms (the standard 1k reservoir and a 60k exact past-task gradient buffer) and at momentum 0.0 and 0.9. Using the exact past-task gradient switches off estimator variance at the source, ensuring that any excursion surviving both the small step size and the exact gradient is purely a property of the path.

The momentum leg provides a secondary perspective. Because momentum multiplies the asymptotic step by $1/(1-\mu)$, the $\mu = 0.9$ rungs operate at effective steps of 1.0, 0.33, 0.1, and 0.01. This allows us to compare exact effective-step diagonals between the two momentum settings. A momentum cell that behaves like its matched $\mu = 0$ partner indicates that momentum's primary effect is bounded by the effective step size, whereas deviations point to the velocity accumulator itself.

Since forward Euler's global error is $O(\eta)$, we estimate the limit by fitting a linear residual metric $A(\eta) = A_0 + k\eta$ over the small-η rungs. A departure from linearity at $\eta = 0.1$ signifies the discrete instability responsible for the first-step spike. The two levels predict opposite behaviors: depth (the overshoot) should fall toward zero down the ladder, while the area in flow time (the arc) should converge on a positive A_0 that no further step-size reduction can eliminate.

Label	η	Epochs c	Cadence	Steps	τ
S1	0.1	1	1	235	23.5
S2	0.1/3	3	3	705	23.5
S3	0.01	10	10	2350	23.5
S4	0.001	100	100	23500	23.5

Table 5.1: The step-size ladder on two-task rot-MNIST. Cadence is the dense post-switch evaluation interval in steps, so every rung records the same 235 samples over T_1. Each rung runs on both buffer arms at both momentum settings: sixteen cells, five seeds, warm-started from one frozen θ_0^* per momentum setting and seed.

5.2.2 The λ-Curriculum Sweep (RQ2)

The curriculum sweep evaluates the scalar feedback gate by testing its components individually (Table 5.2). The central claim of the curriculum is not just that it lowers the performance gap, but that it successfully does so at the standard step size, within the standard step budget, and by modifying only the new-task update. All conditions run on top of standard ER, making the schedule the only independent variable.

Conditions C1 through C3 test linear ramps to evaluate the $O(1/N)$ prediction of Equation (3.2). Condition C4 introduces the adaptive schedule (the feedback variant of Section 3.3), which sets λ from a clipped exponential moving average of $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$, reset to zero at each boundary. Because $\|g_{\text{replay}}\| \approx 0$ at θ_0^*, it starts near zero; unlike the ramps it has neither a length nor a terminal state, tracking the ratio for as long as the task runs and falling again if the ratio does. How far it releases the new task is therefore an outcome of the run rather than a setting, and the two momentum legs differ sharply in it (Appendix A.3). Conditions C5 through C7 apply a minimum floor ($\lambda_{\min}$) to preserve early plasticity. Condition C8 serves as a diagnostic substitution argument, pairing the optimal practical configuration (C7) with the exact 60k replay gradient. If the curriculum leaves a residual transient at momentum 0, applying the exact gradient tests whether that residual is merely buffer noise. Comparing this to momentum runs helps isolate momentum's variance-reduction properties from its role in accelerating new-task learning.

Label	Condition
Baseline	Vanilla ER ($\lambda \equiv 1$)
C1	Linear N=50
C2	Linear N=100
C3	Linear N=200
C4	Adaptive
C5	Adaptive + $\lambda_{\min}$=0.05
C6	Adaptive + $\lambda_{\min}$=0.10
C7	Adaptive + $\lambda_{\min}$=0.20
C8	C7 + full 60k buffer

Table 5.2: λ-curriculum conditions, all applied on top of standard ER and all run at momentum 0.0 and 0.9. The linear conditions ramp λ as $\text{clip}(t/N, 0, 1)$; the adaptive ones use $\max(\lambda_{\min}, \text{clip}(\text{EMA}_\alpha(r), 0, 1))$ with $\alpha = 0.05$ and $r := \|g_{\text{replay}}\|/\|g_{\text{new}}\|$. C4–C7 settle the two ingredient choices of the shipped configuration and are reported in Appendix C.2.

5.2.3 Momentum Cross (RQ2, RQ3)

To test the composability prediction of Section 3.3, we run the vanilla ER reference, the step-size ladder, the curriculum sweep, and the asymmetric-PER sweep at both momentum 0.0 and 0.9 (denoted with an M subscript). This cross-examination reveals whether momentum helps or hinders a method based on where that method attenuates the parameter push. We also conduct a multi-boundary check using a five-task rotMNIST sequence to repeat this three-way comparison across multiple transitions (Appendix C.9).

5.2.4 Asymmetric-PER δ Sweep (RQ3)

The damping sweep (Table 5.3) evaluates the directional feedforward gate by testing how the gap shrinks as the filter strengthens and identifying where the gate ultimately fails. Asymmetric PER implements Equation (3.4), where the gain along an eigendirection of replay curvature ρ is defined as $\delta/(\rho+\delta)$. This means $\delta \to \infty$ recovers plain ER, while $\delta \to 0$ forces the update into replay-flat directions while keeping the restoring gradient at full strength.

Both legs sweep $\delta \in \{1.0, 0.3, 0.1, 0.03\}$, from a filter that barely moves the update to the strongest setting reached here. The standard leg matches the practical memory budget of the curriculum, while the full-buffer leg uses the exact past-task gradient to eliminate variance, allowing us to measure the deterministic arc directly. We include two controls at the settings where the effects they check for would appear: a solver control rerunning the strongest full-buffer cell ($\delta = 0.03$) with 60 conjugate-gradient iterations instead of 25, to rule out numerical artefacts, and a symmetric control over the same four δ on the standard leg that preconditions the summed gradient, isolating the specific effect of the one-sided gate. When momentum is active, the filtered direction enters the velocity accumulator, meaning momentum acts downstream of the gate.

Label	Condition	Replay buffer
P1	Standard leg	1k reservoir
P2	Full-buffer leg	60k (exact g_{rep})
P3	Solver control	60k
P4	Symmetric control	1k reservoir

Table 5.3: Asymmetric-PER conditions. Both sweep legs run at momentum 0.0; momentum-0.9 counterparts cover $\delta \leq 0.3$ on the standard leg and $\delta \in \{0.3, 0.1\}$ on the full-buffer leg. The two controls run at momentum 0, where the effects they check for arise. The filter metric is always the replay Fisher F_{rep}, estimated on a sampled batch (Appendix A.5).

5.2.5 Generalisation: CIFAR-10 (RQ4)

The CIFAR-10 block tests whether the gating mechanisms and momentum dynamics scale to a deeper backbone and a more severe task shift (Table 5.4). We evaluate each method using both BatchNorm and GroupNorm to isolate optimisation dynamics

from normalisation transients. Because the input shift disrupts BatchNorm's running statistics, it forces a readaptation phase during the first few steps. GroupNorm avoids cross-batch statistics entirely, ensuring that its transient reflects pure optimisation behavior.

The curriculum and asymmetric PER carry their tuned rot-MNIST configurations directly into this block without CIFAR-specific hyperparameter sweeps, ensuring a rigorous test of their generalisation. Furthermore, we test the practical curriculum against vanilla ER on novel corruptions (such as a shot-noise shift and a contrast shift) to confirm the results are robust across different types of distribution shifts rather than overfit to Gaussian noise.

Label	Method	Norm	μ
G1	Vanilla ER	BatchNorm	0.9
G2	Adaptive $\lambda_{\min}$=0.20	BatchNorm	0.9
G3	Asym. PER δ=0.1	BatchNorm	0.9
G4	Vanilla ER	GroupNorm	0.9
G5	Adaptive $\lambda_{\min}$=0.20	GroupNorm	0.9
G6	Asym. PER δ=0.1	GroupNorm	0.9
G7	Vanilla ER	GroupNorm	0.0
G8	Adaptive $\lambda_{\min}$=0.20	GroupNorm	0.0
G9	Asym. PER δ=0.1	GroupNorm	0.0

Table 5.4: CIFAR-10 conditions (two-task Gaussian-noise shift, buffer 5,000, five seeds). Top: the headline block at momentum 0.9. Bottom: the momentum cross, pairing momentum-0 runs of all three methods against their momentum-0.9 counterparts at GroupNorm. The generalisation block (novel corruptions and the three-task sequence, three seeds) is described in the text and reported in Appendix C.7.

6 Results

The results section addresses our core research questions by breaking down the forgetting transient into its fundamental components (Section 6.1, RQ1), evaluating the scalar feedback gate (Section 6.2, RQ2) and the directional feedforward gate (Section 6.3, RQ3), synthesising how momentum dictates the choice of gate (Section 6.4), and confirming that both methods scale to a ResNet-18 on CIFAR-10 (Section 6.5, RQ4).

Reading the metrics. To focus on the underlying phenomena, this section prioritises narrative insights over exhaustive statistical breakdowns (full metric tables are available in Appendix C). Gap depth refers to the worst past-task performance drop following a task switch. Area integrates the excursion over time, capturing how long a task remains compromised.

6.1 The Two Levels, Separated (RQ1)

RQ1 investigates whether the performance drop during a task switch is a fundamental property of the trajectory (an unavoidable "arc") or simply an overshoot caused by discrete optimization steps. We answer this by shrinking the learning rate η toward zero, plotting the trajectory over a matched total optimization flow time.

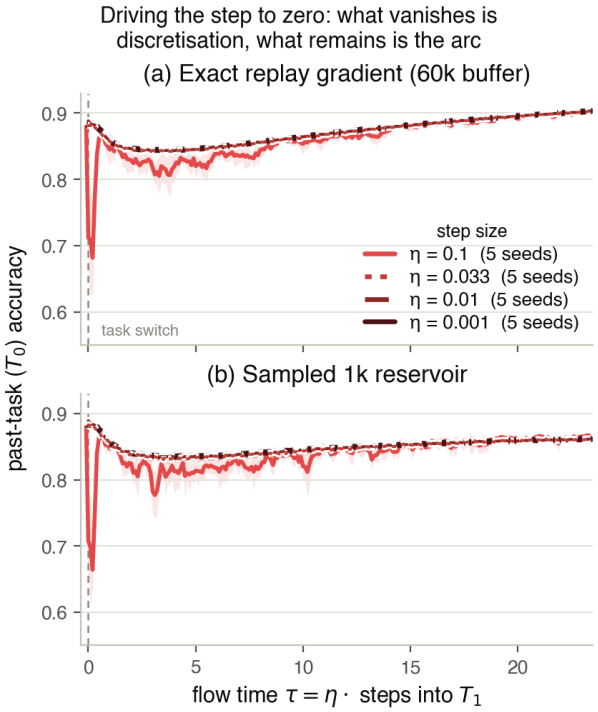


Figure 6.1: The step-size ladder at momentum 0: past-task (T_0) accuracy against flow time $\tau = \sum_t \eta$, seed mean with ±std bands, τ=0 the task switch. Every rung covers the same $\tau = 23.5$ from the same frozen θ_0^* and the same buffer, so within a panel η is the only difference. (a) exact 60k replay gradient, (b) the 1k reservoir of the standard protocol; the legend in (a) applies to both. What the reader should see is a limit that is reached and is not zero. The standard operating point (light, $\eta = 0.1$) leaves the other three within its first two steps, drops ≈20 pp, and spends the rest of the task jittering back; the three finer rungs (dark: $\eta = 0.1/3$ and $\eta = 0.01$ dashed, $\eta = 0.001$ solid) have no boundary spike at all, and overlap so closely that linestyle rather than colour is what separates them. What they do have is a smooth dip-and-recover bend that neither the smaller step nor the exact gradient removes: the arc.

The ladder converges, and not on zero. To understand what drives the gap, we must distinguish between two metrics: the *depth* of the performance drop and the *area* of the excursion. Figure 6.1 illustrates this distinction perfectly. When we reduce the step

size from the standard 0.1 point to much finer resolutions, the massive initial spike vanishes. This dramatic reduction in depth proves that the bulk of standard catastrophic forgetting is a discrete instability, a direct result of a large step size hitting an unbalanced boundary gradient.

However, as the ladder of step sizes converges, it does not converge on zero. Even as the step size vanishes, a smooth, shallow dip remains. This is easiest to see in Figure 6.1(a), where we use an exact replay gradient to eliminate any variance from memory sampling. Even without estimator noise, and with a near-zero step size, the trajectory still registers a positive limit in both its residual depth and its flow-time area. The step size cures the discrete overshoot, but this stubborn, unremovable excursion is the inherent "arc" of the trajectory.

Accuracy is not what the limit costs. Crucially, shedding the discrete instability to reach this minimal arc does not sacrifice final accuracy, the fine-grained runs recover just as well as the standard run. Instead, the cost is purely computational: traversing the exact same trajectory with tiny steps requires drastically more epochs. The challenge for our proposed gates (RQ2 and RQ3) is to achieve this minimal, arc-only forgetting gap at standard, practical step sizes without exploding the compute budget.

6.2 The Scalar Feedback Gate (RQ2)

The scalar feedback gate (adaptive curriculum) successfully protects past knowledge by globally scaling down the learning rate when boundary gradients threaten stability. Table 6.1 puts its practical configuration (C7) beside the directional gate of Section 6.3 and the ungated baseline, at both momentum settings; the ingredient sweep behind C7 is in Appendix C.2.

The schedule works, but incurs a plasticity debt without momentum. As shown in Figure 6.2 (dashed lines), testing the curriculum without momentum reveals the core trade-off. By scaling down the step size during the boundary transition, the gate effectively shrinks the *depth* of the initial forgetting spike and reduces the total *area* of the excursion compared to the vanilla baseline. However, Figure 6.2(b) illustrates the cost: early learning on the new task is visibly delayed, creating a plasticity debt that depresses final accuracy. Furthermore, while the curriculum prevents the catastrophic initial drop, the dashed blue line in Figure 6.2(a) shows it still suffers from a jittery, chronic baseline of forgetting. This is because the curriculum scales the update but cannot fix the underlying variance of the limited replay buffer.

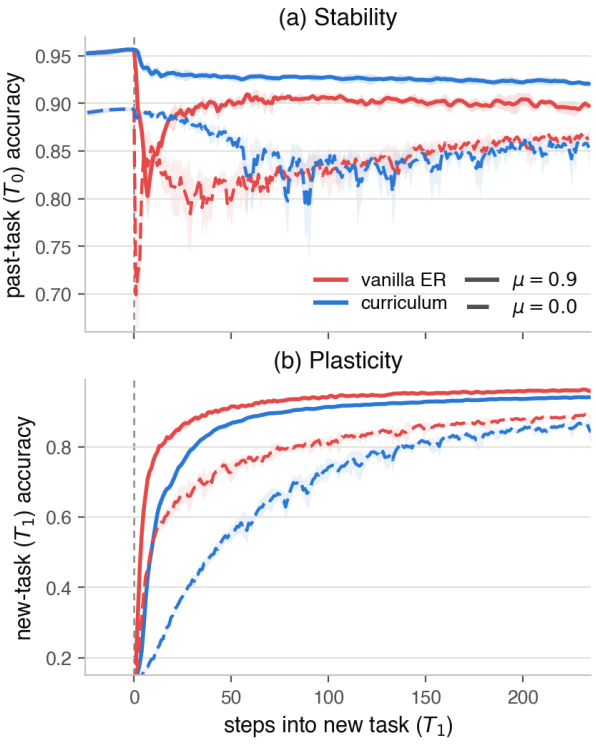


Figure 6.2: The scalar feedback gate and momentum's two roles: per-step accuracy across the rot-MNIST switch, seed mean with ±std bands, for vanilla ER (red) and the practical adaptive curriculum $\lambda_{\min}$=0.20 (blue), each at momentum 0.9 (solid) and 0.0 (dashed). (a) Stability (T_0): only curriculum-with-momentum (solid blue) holds the past task near its plateau through the switch; momentum barely dents vanilla's spike, and both methods open a deep, jittery gap without it (dashed). (b) Plasticity (T_1): at momentum 0 the curriculum slows early new-task learning (dashed blue), the plasticity debt that momentum pays back (solid blue).

Condition	depth (pp)	area	ACC	WP_{100}
Momentum 0.0				
Vanilla ER (no gate)	19.5 ± 4.1	6.27 ± 0.47	0.873 ± 0.005	0.736 ± 0.020
C7: curriculum, $\lambda_{\min}$=0.20	13.9 ± 3.4	2.71 ± 0.57	0.837 ± 0.031	0.710 ± 0.010
P4: asym. PER, δ=0.03	$\mathbf{4.8 \pm 1.9}$	$\mathbf{1.04 \pm 0.58}$	$\mathbf{0.880 \pm 0.005}$	$\mathbf{0.741 \pm 0.007}$
Momentum 0.9				
Vanilla ER (no gate)	15.9 ± 2.6	1.03 ± 0.15	0.928 ± 0.003	0.808 ± 0.012
C7: curriculum, $\lambda_{\min}$=0.20	$\mathbf{3.7 \pm 0.4}$	$\mathbf{0.04 \pm 0.02}$	$\mathbf{0.931 \pm 0.002}$	0.812 ± 0.009
P4: asym. PER, δ=0.03	8.5 ± 0.6	0.23 ± 0.12	0.929 ± 0.003	$\mathbf{0.816 \pm 0.010}$

Table 6.1: The two gates against the ungated baseline on rot-MNIST, at momentum 0.0 and 0.9; five seeds, standard (1k) buffer, best in each block in bold. Each gate is shown at the configuration its own section settles on: the practical curriculum (C7) and the strongest filter (P4). Read down a block and then across the two, the table is the regime flip in four numbers: without momentum the directional gate is far the better of the two and is the only one that reaches its protection without paying accuracy or WP_{100} for it, while the scalar gate buys a smaller gap with both; with momentum the order reverses, the scalar gate reaching 3.7 pp at vanilla's own accuracy while the directional gate re-inflates to 8.5. Areas are on the step axis ($\eta = 0.1$). The full sweeps behind these rows, C1–C7 and $\delta \in \{1.0, 0.3, 0.1, 0.03\}$ on both buffer legs, are in Appendices C.2 and C.4.

Momentum pays the debt and closes the gap. The scalar gate is explicitly designed for a momentum-enabled regime, where it excels by solving both the variance and plasticity problems simultaneously. As seen in the solid lines of Figure 6.2, momentum acts in two complementary roles. First, it averages out the noisy replay estimates over time, collapsing the remaining jittery variance and holding the past task nearly flat against its plateau (Figure 6.2(a)). This drives both the *depth* and *area* of the gap down to a mere fraction of the vanilla baseline. Second, momentum accelerates the learning process, entirely repaying the plasticity debt incurred by the curriculum's early braking (Figure 6.2(b)). Ultimately, the combination of the adaptive curriculum and momentum achieves profound stability while fully restoring final accuracy to vanilla levels as seen in Table 6.1.

6.3 The Directional Feedforward Gate (RQ3)

The directional feedforward gate (asymmetric PER) takes a surgical approach: rather than scaling down the entire update like the scalar curriculum, it selectively filters out only the specific gradient components that align with the high-curvature directions of the past task. Table 6.1 carries it at the strongest filter it was swept to (δ=0.03) on the standard 1k buffer, the operating point of every other experiment; the sweep that gets there, $\delta = 1.0 \to 0.03$ at both momentum settings, is in Appendix C.4.

Protection at no plasticity cost. Figure 6.3 puts the strongest filter against the ungated baseline in the layout the scalar gate was read in, so the two gates can be compared panel for panel. At momentum 0 (dashed) the filter holds the past task some four points above vanilla ER through the entire transition, and (b) shows what it charges for that: the new-task curve lags over roughly the first fifty steps and is indistinguishable from vanilla thereafter. The slowdown is selective, confined to the directions the past-task curvature flags rather than applied to the whole loss, and it is gone by the WP_{100} horizon. This is the scalar gate's plasticity debt inverted, and it is why depth and area fall across the sweep while ACC and WP_{100} *rise* (Table 6.1): the gate recovers the idealised "arc" trajectory without microscopic step sizes and without a plasticity debt. The monotonicity in δ behind the table, on both tasks and at both momentum settings, is Figure C.3; the exact-gradient (60k) diagnostic controls and the late failure they expose are in Appendix C.4.

Momentum re-inflates what the filter brakes. The protection does not survive the accelerator. Repeating the same standard-buffer sweep at momentum 0.9 (the solid lines of Figure 6.3, rung by rung in Figure C.3(c)) moves every rung the wrong way: depth climbs from 7.4, 5.9 and 4.8 pp at $\delta = 0.3$, 0.1 and 0.03 to 10.4, 9.4 and 8.5 pp, the whole ladder displaced upward by 3 to 4 pp with its ordering intact, so no setting of the filter recovers the momentum-0 protection (Table C.5). The mechanism is the per-step locality of the gate. It brakes the push once, at the step it is applied, but momentum accumulates the braked updates and rebuilds the very displacement the filter removed, so what the gate subtracts at one step returns over the next several. Against vanilla ER at the same momentum (15.9 pp) the gate still helps, but the reduction it buys falls from ≈75% of vanilla's depth at momentum 0 to ≈47%, and it buys nothing in accuracy that vanilla does not already have (0.929 against 0.928). The area moves the other way, $1.04 \to 0.23$ at $\delta = 0.03$, as it does for vanilla ER itself ($6.3 \to 1.0$): momentum shortens every transient, so depth rather than area is the axis on which the loss of protection is visible. This is the failure the scalar gate does not

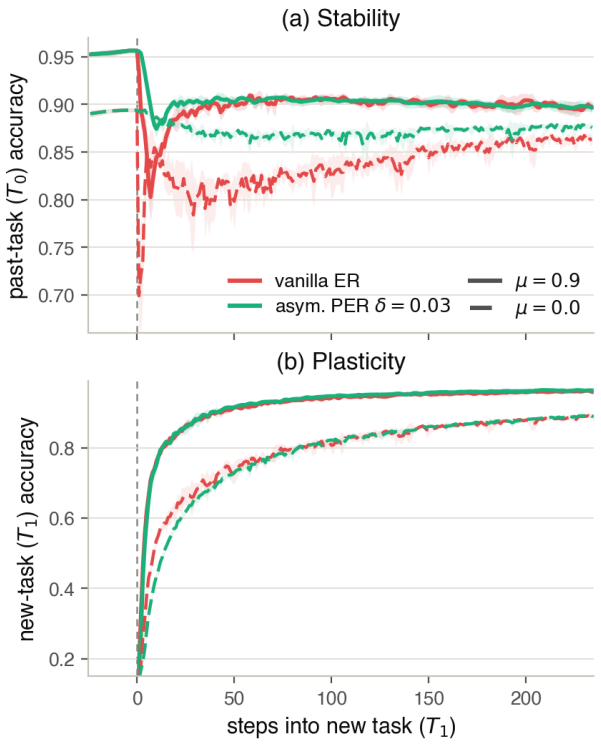


Figure 6.3: The directional feedforward gate and momentum's cost to it: per-step accuracy across the rot-MNIST switch, seed mean with ±std bands, for vanilla ER (red) and the strongest filter, asymmetric PER at δ=0.03 (aqua), each at momentum 0.9 (solid) and 0.0 (dashed). (a) Stability (T_0): without momentum the filter all but removes vanilla's spike and the chronic dip behind it; with momentum the same filter is re-inflated, its dip returning to within a few points of vanilla's, because the accumulator rebuilds over the following steps the displacement a per-step brake removes. (b) Plasticity (T_1): the filter lags for roughly fifty steps and then tracks vanilla at both momentum settings, the selective slowdown that the scalar gate's global braking is not (Figure 6.2(b)).

share, and it is what Section 6.4 reads as a regime flip rather than a trade-off.

6.4 Reading the Momentum Crosses Together

When evaluated side-by-side on realistic memory budgets, it becomes clear that the two gates do not sit on a continuous stability-plasticity trade-off curve. Instead, their viability is strictly dictated by the momentum regime, and the two blocks of Table 6.1 are that flip read off one set of columns.

Without momentum, the directional gate is vastly superior: it drastically reduces the *depth* of the forgetting gap and keeps the past task stable, whereas the scalar curriculum opens a deep, noisy dip. However, in standard deep learning configurations where momentum is active, the hierarchy completely flips. Accumulated velocity cripples the directional gate, causing its forgetting *depth* to swell back up as the filtered gradients re-inflate. Conversely, the scalar curriculum leverages momentum to smooth out its variance, holding the past task virtually flat while fully restoring final accuracy. The choice is binary: use the directional gate for unaccelerated optimisers, and the scalar gate when momentum is enabled.

6.5 Generalisation to CIFAR-10 (RQ4)

To ensure these findings are not artifacts of simple environments, we tested both gates on a ResNet-18 architecture transitioning across CIFAR-10 corruptions. Table 6.2 and Figure 6.4 confirm that the mechanisms scale robustly to deeper networks.

A rung short on the directional gate. One configuration mismatch should be stated before the numbers are read: the directional gate is run here at δ=0.1, one rung weaker than the δ=0.03 that Section 6.3 settles on and that Table 6.1 reports, so this block does not carry the shipped setting. The two rungs are adjacent on a sweep that is monotone in δ at both momentum settings, and on the momentum-0.9 leg these runs share they are close: on rot-MNIST they differ by 0.9 pp of depth (9.4 against 8.5) at accuracies and plasticity horizons that do not separate at all (ACC 0.928 against 0.929, WP_{100} 0.815 against 0.816; Table C.5). The stronger filter is the better one wherever the sweep was run, so these rows are if anything a slight understatement of what the shipped setting would give, and the difference between the two rungs is far smaller than any effect this section turns on. It remains a gap in the protocol rather than a result: what generalises here was tested one rung short.

Scaling to deep backbones. Both gates successfully protect past knowledge in harder tasks, though

Norm	Method	gap_depth (pp)	gap_area_end	ACC	min-ACC
BatchNorm	G1: Vanilla ER	11.8 ± 2.5	17.5 ± 4.6	0.723 ± 0.008	0.644 ± 0.023
	G2: Curriculum ($\lambda_{\min}$=0.20)	$\mathbf{4.8 \pm 0.7}$	$\mathbf{3.6 \pm 2.2}$	0.722 ± 0.011	0.713 ± 0.017
	G3: Asym. PER (δ=0.1)	5.9 ± 4.3	3.3 ± 3.0	0.715 ± 0.011	0.698 ± 0.023
GroupNorm	G4: Vanilla ER	41.2 ± 13.5	34.9 ± 23.7	0.730 ± 0.011	0.309 ± 0.111
	G5: Curriculum ($\lambda_{\min}$=0.20)	$\mathbf{5.2 \pm 2.1}$	$\mathbf{1.7 \pm 0.9}$	0.731 ± 0.020	0.670 ± 0.055
	G6: Asym. PER (δ=0.1)	5.6 ± 1.1	4.8 ± 3.8	0.741 ± 0.009	0.673 ± 0.023

Table 6.2: CIFAR-10 headline block (ResNet-18, momentum 0.9, buffer 5,000, five seeds). Both gates cut gap depth and area far below vanilla ER at matched final accuracy and much higher worst-case retention (min-ACC), under both normalisations. Neither was given a CIFAR-specific sweep. The curriculum carries its rot-MNIST configuration; the directional gate is run one rung weaker than the δ=0.03 of Table 6.1 (see text).

their visual impact varies by normalisation layer. Under batch normalisation (Figure 6.4(a)), the vanilla baseline suffers a moderate drop in *depth* and accumulates a large forgetting *area* as it slowly re-adapts. Both gates intercept this, effectively freezing the past task near its peak performance.

The contrast is drastically sharper under group normalisation (Figure 6.4(b)). Here, the vanilla baseline suffers a catastrophic, prolonged crash, heavily inflating both the gap *depth* and *area*. Both the scalar and directional gates effortlessly suppress this massive transient, keeping the trajectory virtually flat through the switch. As Table 6.2 shows, this intervention vastly improves worst-case task retention (*min-ACC*) across the board without sacrificing final *accuracy*.

The deciding factor is compute cost. Since both methods successfully minimise the gap *depth* and *area* during the task transition, the deciding factor in deep networks becomes efficiency. Because the directional gate requires per-step conjugate-gradient solves to compute the filter, it consumes substantially more wall-clock time and GPU energy. The scalar gate, requiring only a single gradient-norm ratio, provides equivalent stability at a fraction of the computational footprint.

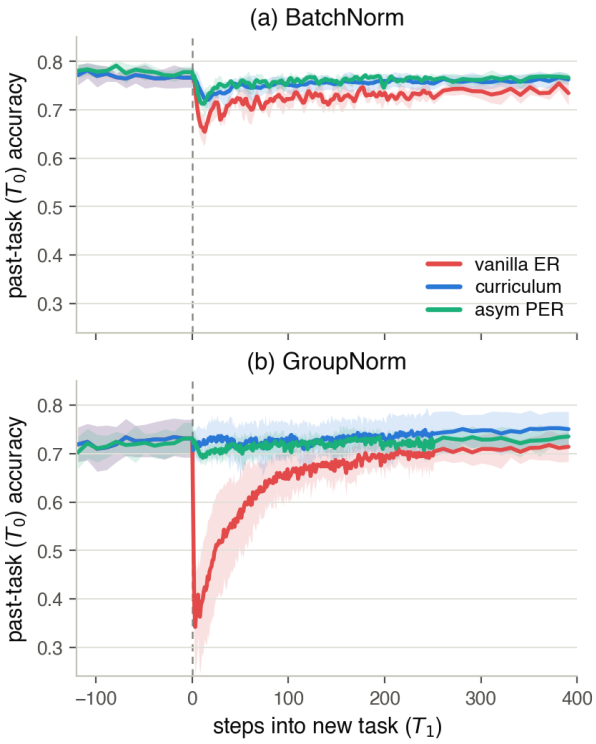


Figure 6.4: Per-step past-task (T_0) accuracy across the CIFAR-10 transition (ResNet-18, momentum 0.9, buffer 5,000), one panel per normalisation, each overlaying vanilla ER (red), the practical curriculum (blue) and asymmetric PER (aqua); seed mean with ±std bands, x=0 the task switch. (a) BatchNorm: vanilla ER dips and re-adapts slowly while both gates hold T_0 near its plateau. (b) GroupNorm: vanilla ER's transient is deep and long, whereas both gates hold T_0 flat through the switch (Table 6.2, Appendix C.6).

7 Discussion

The results support the two-level account and, with it, the gate view of replay. This section reads them back against the theory, weighs the limits of the evidence, and draws out what follows for method design.

7.1 Why the Gap Needs Two Levels

The central claim is conceptual, and the experiments support it: the stability gap is not one quantity but two deviations of the same optimisation, kept apart because they answer to different interventions. The step-size ladder of Section 6.1 is the direct test, because a uniform rescaling of η is a time reparametrisation of the same gradient flow and is therefore the one

intervention that changes how faithfully the intended trajectory is followed without changing which trajectory that is. Neither kind of intervention can substitute for the other, and the ladder makes one half of that statement a limit rather than a comparison: with both Level-Two drivers switched off at once, the step driven towards zero and the replay gradient exact, what is left is not zero but the arc. The converse holds at the standard operating point, where the curriculum replaces the path and the overshoot stays standing behind it; the transient only closes almost completely when the path repair is joined by the removal of the Level-Two driver it leaves untouched, the estimator variance of the finite buffer (C8, Section 6.2). The ladder also settles what the boundary imbalance is, which a driver reading would have placed on the overshoot side. Rescaling η leaves the vector field untouched, so the imbalance at θ_0^* is identical at every rung, the same vanishing replay gradient meeting the same large new-task gradient; yet the spike it produces at the standard step is gone at a third of that step, and what remains is smooth. The imbalance is Level-One terrain and the step size that meets it is the acute driver. At the same time, no intervention here is pure. The curriculum, nominally a path intervention, also shrinks the early push and with it the overshoot. An intervention aimed at one level can therefore move the other; what it cannot do is replace an intervention at the other level, and that non-substitutability is the practical content of the separation.

The natural alternative, a single additive decomposition of the gap into magnitude, variance, and trajectory shares, does not survive the data. The shares are order-dependent (a noisy replay gradient inflates the first-step magnitude), the "magnitude" share dissolves once the step size that converts it into a drop is taken away, and the mapping from factors to metrics is crossed by the experiments, the curriculum's momentum-0 tail looking like a "trajectory" term until C8 removes it with the exact buffer and no momentum at all, revealing it as variance (Table C.3). The two-level split is instead by kind: which path is followed versus how tightly it is followed.

The split also separates two costs that usually travel together. C8 closes the transient almost completely while its final accuracy sits well below vanilla ER's (Section 6.2). The permanent rise of the past-task loss to its joint level is the stability–plasticity cost, and it remains after the transient is engineered away; the dip below that level and back, the part continual evaluation reveals, is a failure to follow the reference path, and the C8 cell shows it is removable. The gap is the second cost, not the first: it is not the price of learning the new task.

Finally, the two levels explain why the metric that matters changes with the backbone. On the MLP the network reconverges in a few steps, so the arc integrates to little and the perceived severity is the overshoot spike; on the ResNet the same transient stays open for hundreds of steps (Figure 6.4), so the slow tail is where most of the lost accuracy accumulates and where both gates' margin over vanilla is largest (Table 6.2). Depth and area are the signatures of the two levels, and a depth-only reading would have understated the result on exactly the architectures where continual-evaluation cost is highest.

7.2 Two Gates, Two Ways to Divide the Work

In the unified update of Section 3.3, the curriculum and asymmetric PER occupy the taxonomy's two pure corners: a scalar gate driven by feedback, and a directional gate driven by feedforward. The results show the two corners working and failing exactly where the taxonomy says they should, and the single fact that organises the whole comparison is *where* the gate attenuates the push.

The scalar feedback gate attenuates at source: it multiplies the entire new-task loss by a schedule that starts near zero, so early in the transition there is almost no push to discretise, accumulate, or let noise act on. Three consequences follow, and the data confirm each. It composes with momentum, because the accumulator never sees a sustained full-strength push to rebuild (Section 6.2). Read inside the schedule alone that composition would be indistinguishable from momentum's general convergence gain; what turns the mechanistic reading into an observation is the contrast with the directional gate, which attenuates the same push at the step and whose protection the same accelerator undoes (Section 6.3). The two gates have little in common beyond the one-sided form and the gradient they attenuate, yet momentum moves them in opposite directions, and the direction follows where the attenuation lands. It has no curvature blind spot, because the adaptive schedule reads a live damage signal rather than a model of the past task. And it pays a plasticity debt: holding the loss down slows the new task, a debt only acceleration repays (momentum restores C8's final accuracy, Table C.3). The scalar gate's price is plasticity, and momentum is how it is paid.

The directional feedforward gate attenuates per step: it filters the push through the replay Fisher, slowing the new task only along the directions the past-task curvature flags rather than scaling the whole loss down, and protects the past task at no measured plasticity cost, ACC and short-horizon plasticity rising as the filter strengthens (Table C.5). The same per-step locality is its weakness on two axes, one the taxonomy predicts and one that emerged unanticipated. Under momentum the protection unravels by the re-inflation mechanism of Section 6.3, depth climbing from 4.8 to 8.5 pp on the standard buffer: the same lever that closes the scalar gate's gap re-opens

the directional gate's, and the difference is entirely where the attenuation is applied, at the source or at the step. That one distinction accounts for the whole momentum column of the study: the vanilla null, the curriculum's composition, and the directional gate's re-inflation.

The directional gate's second failure was not predicted, and it is the sharper one because it makes the case for feedback. At the strongest filter the past-task curve can slide off a late cliff, in the exact-gradient control where nothing perturbs the iterate (the δ=0.03 full-buffer cliffs, Appendix C.4). A plausible reading is a blind spot of its curvature model: the sampled Fisher rates a direction flat that the true past-task loss punishes further out, and new-task pressure escapes through it. The exposure is characteristic of feedforward control, whose gate can only be as good as its model of the past task; a feedback gate instead reads the damage as it arrives, and its own weakness lies in the conditioning of the signal it reads, as the three-task sequence shows (Section 7.4). The cliffs and the plasticity debt are thus the two gates' defining and complementary costs. The feedforward gate protects without holding the whole new task back, but it is only as good as its local curvature model of the past task and can fail where that model is wrong. The feedback gate cannot be fooled that way, because it reads the damage itself, but it pays by holding the whole new task back, a debt only an accelerator repays. This is why the two flip with momentum rather than sitting on a fixed frontier (Section 6.4). Both gates carry to the ResNet at matched accuracy and much improved worst-case retention (Table 6.2), though the momentum cross there is confounded (Section 7.4); it is the gate principle, not either particular method, that generalises.

The practical recommendation follows from that momentum column rather than from a frontier. With momentum off, the directional gate is the better of the two on both axes. With momentum on, which is the standard setting on realistic backbones and the one decisive for final accuracy at these budgets, the scalar gate matches the directional gate's retention at essentially vanilla-ER cost, without the blind-spot cliffs and without the $\sim$16$\times$ wall-clock and $\sim$24$\times$ energy overhead of a per-step Fisher solve (Table C.10). C7$_M$, the adaptive schedule with $\lambda_{\min} = 0.20$ under momentum, is accordingly the configuration this thesis recommends and the one carried to CIFAR.

7.3 What the Account Changes in the Literature

The two-level split lets the existing accounts of the gap compose rather than compete: the boundary imbalance of De Lange et al. [4], the persistence under the exact joint loss shown by Hess et al. [12], and the geometry Kao et al. [14] derives for it are observations about different parts of one mechanism, and the ladder assigns each its place. The imbalance is a feature of the intended trajectory and by itself does not produce the measured drop: it is identical at every rung, yet the spike is gone at a third of the standard step (Section 6.1). What converts it into that drop is the step size that meets it, which Obis [21] reach from the optimiser side. The persistence under exact gradients is the arc, measured on the ladder's exact arm rather than inferred. The decomposition thus connects the imbalance account to the measurements through the step size, and situates the geometry account as the part no repair of the step reaches.

The unified update of Sodhani et al. [24] sorts replay methods by which weight they move and from what signal; the results add the axis that turned out to be decisive, where the modulation is applied. A-GEM and MEGA correct the update per step from a live signal [2, 9], which places them on the step side of that divide, the side whose protection momentum unwinds (Section 6.3). The account therefore predicts that their boundary protection weakens under momentum while source-attenuating schedules compose with it. These results do not test that prediction, but it is directly testable, and the original comparisons predate the continual-evaluation lens that would have registered per-step protection eroding.

7.4 Limitations

Several limits bound these claims, beginning with the statistical one: with five seeds the design cannot reach a 0.05 threshold (Section 5.1.2), so the argument rests on effect sizes and seed-paired direction. The effects the story leans on, the up-to-eightfold depth reductions on CIFAR, the C7$\rightarrow$C8 variance collapse, the momentum re-inflation of the directional gate, are large relative to the seed spread and unanimous across seeds; the smaller ones, the adjacent-δ steps and the $\lambda_{\min}$ trade-offs, are read as directional consistency only, and one, the adaptive-versus-best-linear depth, does not reach even that ($p = 0.44$).

What the ladder settles at momentum 0 it settles under momentum too, but the two answers are not directly comparable. Momentum places its leg a decade coarser in effective step, so only the fourth rung reaches the range in which the momentum-0 leg had already converged, and there it does flatten onto an arc of its own (Section 6.1). Because each leg warm-starts from an anchor trained at its own momentum and the two settle some seven accuracy points apart, the difference between their limits is a difference between two operating points rather than a measurement of what momentum does to the arc; isolating that would need a shared anchor this design does not provide. The arc's magnitude is also one measurement in one place, two-task rot-MNIST on an MLP from a single anchor θ_0^* per momentum setting and

seed. That the excursion converges on a positive limit is a structural claim the design supports; how large the arc is on another benchmark or backbone these runs cannot say, and the ResNet's far longer transient (Figure 6.4) suggests the balance between the two levels is itself architecture-dependent. Nor is the ladder's own remedy a method. Matched flow time makes the epoch count scale as $1/\eta$, so the rung at which the momentum-0 leg converges costs ten times the training and the momentum leg needs a further decade below it; the shorter step buys the separation and is a diagnostic, not a technique to be adopted in place of a gate.

The gates carry limits of their own. The scalar gate's accuracy-neutrality is a property of the full curriculum-plus-momentum configuration rather than of the ramp: at momentum 0, lengthening the linear ramp lowers depth but costs several points of final accuracy (Table C.2). Its feedback signal has a reliability boundary, the three-task group-norm sequence in which the adaptive ratio loses conditioning at a transition between two near-identical corruptions and the schedule underperforms its control with high seed variance; we flag that as a property of the gradient-ratio signal, absent from the batch-norm variant, rather than of the gate principle, and Appendix C.7 traces it step by step through the logged norms. For the directional gate the solver control establishes that the cliffs are real and not a numerical artefact, but the blind-spot mechanism we offer for them is a reading consistent with the data, not an established cause.

The deep-backbone evidence is thinner in one specific place. The CIFAR momentum cross is uninformative in both directions: at momentum 0 the ResNet under-converges, both gates improve with momentum by similar margins, and the vanilla null sits on that same degraded baseline, so no cell of it cleanly confirms or refutes a momentum signature. All three momentum signatures therefore rest on the rot-MNIST results; a clean deep-backbone test would require a converged momentum-0 baseline we did not run.

Two scope limits are worth stating plainly. The exact-gradient conditions (the ladder's 60k arm, C8, and the full-buffer leg of the δ sweep) store the entire past task and so break the limited-memory premise of continual learning; they are diagnostic probes that isolate the arc and split momentum's roles, and the practical configurations are the 1k-buffer curriculum and the standard-buffer directional gate. And the directional gate's per-step Fisher-vector solve adds a fixed multiple of the backward cost per step, $\sim$16$\times$ the wall-clock and $\sim$24$\times$ the GPU energy of the curriculum on the ResNet-18 (Table C.10), an overhead the scalar gate's single gradient-norm ratio does not carry. The headline analysis also rests on a single-transition, two-task design chosen to isolate one boundary; the five-task rot-MNIST sequence (Appendix C.9) shows the regime picture surviving four consecutive boundaries, but a systematic study of longer sequences and of class-incremental rather than domain-incremental shifts is left open.

7.5 Outlook

The taxonomy that organises the two methods also names what is missing from it. The gates studied here fill three of its four corners: scalar×feedback, directional×feedforward, and, as an ablation, the linear ramp at scalar×feedforward. The remaining corner, a directional gate driven by a live damage signal rather than by a precomputed curvature model, is the natural candidate the two failure modes point to: it would filter the push by direction, keeping the plasticity that makes the feedforward gate cheap, while reading the damage as it arrives rather than trusting a model of the past task. Whether one design can deliver both properties at once is an open question, not an implication of these results, but the momentum results constrain any attempt in advance: its directional attenuation should act at source or on the accumulated velocity, not on the instantaneous step, or it inherits the re-inflation of Section 6.3.

Four narrower directions follow from the failure modes and costs.

- *Non-uniform step sizes.* The ladder maps the uniform step-size axis and maps it to a limit; what stays open is where per-parameter adaptive step sizes sit in the gate taxonomy [21], and whether an attenuation that is directional in the coordinate basis but applied to the step inherits the re-inflation the directional gate suffers under momentum. The second is the sharper question, because these optimisers carry an accumulator of their own.

- *Re-inflation in existing methods.* Any approach that preconditions the update with past-task curvature per step, the natural-gradient family included [14], carries the same exposure, and moving its attenuation to the accumulated velocity is a directly testable fix.

- *Cheaper directional gates.* The per-step conjugate-gradient solve could be replaced by a diagonal or Kronecker-factored Fisher, refreshed as coarsely as once per task boundary as Natural Continual Learning does [14]. Whether the protection survives the coarser metric or the staler one is open; on backbones much larger than the ResNet-18 here such an approximation is a precondition for the directional corner rather than a refinement of it.

- *Other variance reducers.* Momentum's gap-closing role proved interchangeable with source variance reduction (C8), so the curriculum could

be paired with alternatives; iterate averaging, which carries none of momentum's accumulator-driven overshoot, is the natural candidate.

The cliffs add a methodological rule. They arrive late and abruptly, after long stretches of exemplary retention, and end-of-task metrics would have recorded the δ=0.03 runs as successes; only per-step measurement caught the failure. Continual evaluation [4] is usually presented as the lens that reveals the stability gap. These results argue for it also as an acceptance test for any feedforward gate deployed in a system that serves predictions while it trains: a gate that acts on a model of the past task can fail precisely where that model is wrong, silently until it does.

Statement on the Use of AI Tools

In accordance with the University of Groningen's guidelines on the responsible use of generative AI, I disclose that AI assistants were used during the preparation of this thesis. The tools were Google's Gemini 3.x series, Anthropic's Claude 4.x series, and Anthropic's Claude Fable 5, the releases current across the period, February to July 2026, in which the work was carried out. They functioned as assistive tools under my continuous direction, not as autonomous contributors. Specifically, I used them to brainstorm and stress-test ideas and as a sounding board for reasoning through arguments and explanations; to draft and revise prose, which I subsequently edited for accuracy, precision, and voice; to assist in writing and refactoring the analysis and experiment code, all of which I reviewed, ran, and tested; and to produce first-pass summaries of the literature, which I verified against the primary sources.

The tools were never given open-ended control: every request specified a concrete, bounded task, and all output was reviewed and, where retained, corrected, understood, and integrated by me. All experimental results, figures, and numerical claims were checked against the underlying run data, and all cited work was read in the original. The research questions, experimental design, interpretation of the results, and the overall argument of the thesis are my own, and I take full responsibility for its content, including any errors.

References

[1] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1903.08671. 3, 4

[2] Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-gem. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1812.00420. 4, 17

[3] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On tiny episodic memories in continual learning, 2019. URL https://arxiv.org/abs/1902.10486. 4

[4] Matthias De Lange, Gido van de Ven, and Tinne Tuytelaars. Continual evaluation for lifelong learning: Identifying the stability gap. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2205.13452. 3, 4, 6, 8, 9, 17, 19, 24

[5] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht. Essentially no barriers in neural network energy landscape, 2019. URL https://arxiv.org/abs/1803.00885. 5

[6] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis, 2020. URL https://arxiv.org/abs/1912.05671. 5

[7] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikhin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL https://arxiv.org/abs/1802.10026. 5

[8] Yiduo Guo, Jie Fu, Huishuai Zhang, Dongyan Zhao, and Yikang Shen. Efficient continual pretraining by mitigating the stability gap, 2024. URL https://arxiv.org/abs/2406.14833. 4

[9] Yunhui Guo, Mingrui Liu, Tianbao Yang, and Tajana Rosing. Improved schemes for episodic memory-based lifelong learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/1909.11763. 4, 17

[10] Md Yousuf Harun and Christopher Kanan. Overcoming the stability gap in continual learning. In *Conference on Lifelong Learning Agents (CoLLAs)*, 2024. URL https://arxiv.org/abs/2306.01904. 4

[11] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common

corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1903.12261. 8, 21

[12] Timm Hess, Tinne Tuytelaars, and Gido M. van de Ven. Two complementary perspectives to continual learning: Ask not only what to optimize, but also how. In *Conference on Lifelong Learning Agents (CoLLAs)*, 2024. URL https://arxiv.org/abs/2311.04898. 3, 4, 17

[13] Sandesh Kamath, Albin Soutif-Cormerais, Joost van de Weijer, and Bogdan Raducanu. The expanding scope of the stability gap: Unveiling its presence in joint incremental learning of homogeneous tasks, 2024. URL https://arxiv.org/abs/2406.05114. 4

[14] Ta-Chu Kao, Kristopher T. Jensen, Gido M. van de Ven, Alberto Bernacchia, and Guillaume Hennequin. Natural continual learning: success is a journey, not (just) a destination. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2106.08085. 3, 4, 6, 7, 17, 18

[15] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114 (13):3521–3526, March 2017. ISSN 1091-6490. doi: 10.1073/pnas.1611835114. URL http://dx.doi.org/10.1073/pnas.1611835114. 4

[16] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.08840. 4

[17] James Martens. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020. URL http://jmlr.org/papers/v21/17-678.html. 23

[18] Michael Mccloskey and Neil J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24:104–169, 1989. 3

[19] Martial Mermillod, Aurélia Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in psychology*, 4:504, 08 2013. doi: 10.3389/fpsyg.2013.00504. 3

[20] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Dilan Gorur, Razvan Pascanu, and Hassan Ghasemzadeh. Linear mode connectivity in multitask and continual learning. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2010.04495. 5

[21] Chris Obis. Reaching for resilience: Understanding how optimizers affect the stability gap in continual learning. Bachelor thesis, Delft University of Technology, 2025. URL https://repository.tudelft.nl/file/File_356b427b-c259-4ad5-b47c-16cc7c234716. Supervised by T. Viering and G. M. van de Ven. 4, 6, 17, 18

[22] Roger Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97:285–308, 04 1990. doi: 10.1037/0033-295X.97.2.285. 3

[23] Nicol N. Schraudolph. Fast curvature matrix-vector products for second-order gradient descent. *Neural Computation*, 14(7):1723–1738, 2002. doi: 10.1162/08997660260028683. 23

[24] Shagun Sodhani, Mojtaba Faramarzi, Sanket Vaibhav Mehta, Pranshu Malviya, Mohamed Abdelsalam, Janarthanan Janarthanan, and Sarath Chandar. An introduction to lifelong supervised learning, 2022. URL https://arxiv.org/abs/2207.04354. 3, 4, 17

[25] Albin Soutif-Cormerais, Antonio Carta, and Joost Van de Weijer. Improving online continual learning performance and stability with temporal ensembles, 2023. URL https://arxiv.org/abs/2306.16817. 4

[26] Edoardo Urettini and Antonio Carta. Online curvature-aware replay: Leveraging 2nd order information for online continual learning, 2025. URL https://arxiv.org/abs/2502.01866. 7

[27] Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. *Continual learning and catastrophic forgetting*, page 153–168. Elsevier, 2025. ISBN 9780443157554. doi: 10.1016/b978-0-443-15754-7.00073-0. URL http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0. 4

[28] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Trans. Math. Softw.*, 11(1):37–57, March 1985. ISSN 0098-3500. doi: 10.1145/3147.3165. URL https://doi.org/10.1145/3147.3165. 4, 22

A Implementation Details

This appendix records the implementation specifics that the methods of Section 5 depend on but that would interrupt the argument in the main text: the datasets, the backbones and training protocol, the experience-replay variants, and the asymmetric-PER solver. The full implementation is available at `https://github.com/Arthurreuss/ValleyWalk`.

A.1 Datasets

rot-MNIST rotates each 28×28 image by the per-task angle and feeds the flattened greyscale vector to the MLP. The CIFAR-10 domain shifts are generated with the `imagecorruptions` package, which implements the standard common-corruptions benchmark of Hendrycks and Dietterich [11]; the Gaussian noise, shot noise, and contrast shifts are applied at severity 3 on the benchmark's 1–5 scale, per sample at load time, followed by the standard CIFAR-10 channel normalisation. The clean task ("none") applies the normalisation only.

A.2 Models and Training

Tables A.1 and A.2 give the two backbones, and Table A.3 the training protocol shared across both benchmarks.

Component	Value
Input	784 (flattened 28×28 greyscale)
Hidden layers	2×400 (fully connected, ReLU)
Output	10-class logits
Regularisation	none
Total parameters	478,410

Table A.1: MLP backbone used for the rot-MNIST sweeps.

Component	Value
Input	$3 \times 32 \times 32$ RGB
Stem	3×3 conv, stride 1, no max-pool (CIFAR adaptation)
Stages	4 residual stages, BasicBlocks $[2, 2, 2, 2]$, widths $[64, 128, 256, 512]$
Normalisation	BatchNorm (default); GroupNorm ($\min(32, C)$ groups) variant crossed in Section 5.2.5
Head	global average pool + linear, 10-class logits
Total parameters	≈11.2M

Table A.2: CIFAR-adapted ResNet-18 backbone used for the CIFAR-10 block. The 3×3 stride-1 stem and removed max-pool keep the feature map at 32×32; all other details follow the ResNet-18.

Hyperparameter	rot-MNIST (MLP)	CIFAR-10 (ResNet-18)
Epochs per task	1 (online); c in the ladder	10
Batch size	256	256
Optimiser	SGD	SGD
Learning rate	0.1; $0.1/c$, $c \in \{1, 10, 33\}$, in the ladder	0.1
Momentum	0.0 or 0.9	0.9 (0.0 in cross)
Weight decay	0.0	0.0
Replay buffer	1,000 or 60,000	5,000
Seeds	5	5 or 3

Table A.3: Training protocol for both benchmarks.

The per-step plotting convention. Test accuracy is logged every 10 steps away from a transition and every step inside the dense window (Section 5.1.1), so a short interval separates the last pre-switch evaluation from the first post-update one. In the per-step figures the last pre-switch level is held flat up to the boundary rather than interpolated across that interval: the past task is still training and sitting at its plateau there, so a straight line between the two samples would draw the drop as beginning before the switch that causes it. The convention

affects the drawn curve only. Every reported metric is computed from the logged samples themselves, so none of them depends on it.

A.3 Experience replay

The replay buffer uses reservoir sampling [28], so the stored set is a uniform sample of the past stream at any budget. Two modes appear in the experiments:

- *Standard ER* (the D1 reference, the ladder's 1k arm, and all C-series conditions except C8): each step takes one combined backward pass on $L_{\text{current}} + L_{\text{replay}}$, with half the mini-batch drawn from the current task and half sampled with replacement from the buffer.

- *Full-data replay* (the ladder's exact arm, C8, and the P2–P3 legs): the replay gradient is computed on *every* stored sample exactly once per step (no sub-sampling). Paired with a buffer equal to the full past-task training set (60,000 for rot-MNIST T_0), this yields the deterministic empirical past-task gradient at the current parameters, removing estimator variance.

The λ-curriculum re-weights only the current-task term, minimising $L_\lambda = L_{\text{replay}} + \lambda(t)\, L_{\text{current}}$; the adaptive schedule reads $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$ through an exponential moving average ($\alpha = 0.05$) and applies the optional floor $\lambda_{\min}$. The EMA is reset to zero at each task boundary, so λ starts near zero on every task; the time-indexed schedules latch at $\lambda = 1$ once the ramp is past, but the adaptive one has no such state and follows the ratio for the whole task.

Reconstructing λ from the logged gradient norms of the C4 runs shows how far the two momentum legs actually get, and they get nowhere near the same place. At momentum 0 the norms do converge over the epoch, the median $\|g_{\text{new}}\|/\|g_{\text{replay}}\|$ falling from $\approx$7 over the first twenty steps to $\approx$1.1 over the second half, and λ follows it to 0.79–0.93 at the end of the task; it reaches 1 in one seed of five and then only at step 208 of 235, so even here the release is incomplete when the task ends. At momentum 0.9 the ratio instead widens, from $\approx$10 early to $\approx$40 late, and λ peaks at 0.12–0.16 around step 60 and decays to $\approx$0.02: the gate shuts again rather than opening. With the floor of C5–C7 in place this shows up as saturation, C7_M sitting at $\lambda = \lambda_{\min}$ for 73–79% of the task's steps. The schedule is a feedback rule with no terminal state, not a ramp with an endpoint, and the momentum-on results of Section 6.2 are obtained in the regime where it holds the new-task term down throughout.

A.4 The step-size ladder

Every cell of the ladder warm-starts from a shared task-0 checkpoint. `init.checkpoint` loads a task-0 `state_dict` written by a separate anchor run, one per momentum setting and seed, trained at the standard operating point ($\eta = 0.1$, one epoch); `init.skip_tasks` then suppresses the training loop of that many leading tasks while still running their `end_task` bookkeeping, so the replay buffer is filled from the skipped task's stream exactly as it would have been in an ordinary run. Because reservoir sampling at a fixed seed returns the same subset of the same stream at a given budget, θ_0^* and the buffer are identical across the three rungs at a given buffer arm, momentum, and seed, and the learning rate is the only difference between them.

A skipped task still advances `global_step` by one nominal epoch ($\lceil 60{,}000/256 \rceil = 235$ batches), so the task boundary carries the same index as in an ordinary run and the per-step plotting convention above keys off it unchanged. One pre-switch evaluation is written at step 234, supplying the flat pre-boundary segment that the suppressed training loop would otherwise have recorded.

Inside the dense post-switch window the stability-gap cadence fires on the step index *within the current task* rather than on the global step. At a cadence of $c > 1$ a global-step modulus would place the first post-switch sample at an arbitrary phase offset into the task whenever c does not divide the 235-batch epoch, skipping exactly the steps the boundary spike occupies. The change is a no-op at cadence 1, which is every previously archived run.

The four rungs use $\eta = 0.1$, $\eta = 0.1/3 = 0.0\overline{3}$, $\eta = 0.01$ and $\eta = 0.001$. The second is written into the configuration at full precision rather than rounded, so that $\eta c = 0.1$ holds exactly and the flow time $\tau = 23.5$ is matched to the last digit; the main text reports it as 0.1/3. The dense window is set to $250c$ steps, which exceeds the $235c$ steps of the task, so every cell records the whole of T_1 at the dense cadence: 235 samples per rung, evenly spaced in flow time. Buffer-fidelity diagnostics are disabled throughout the ladder, since each such step costs a full pass over the past-task training set and the longest rung takes thirty-three epochs.

A.5 Asymmetric PER

How the Fisher is computed. For a softmax cross-entropy model the Fisher information matrix equals the generalised Gauss–Newton matrix

$$F = \frac{1}{B}\sum_b J_b^\top H_b J_b, \qquad J_b = \frac{\partial z_b}{\partial \theta}, \qquad H_b = \mathrm{diag}(p_b) - p_b p_b^\top,$$

where z_b are the logits of sample b and $p_b = \mathrm{softmax}(z_b)$ [17]. This matrix is positive semi-definite by construction, so $F + \delta I$ is positive-definite and the solve is well-posed; it depends only on the model's predictive distribution, not on the labels. F_{rep} is estimated on a mini-batch sampled from the replay buffer and *recomputed at every step from the current parameters*: the gate uses a live local curvature model, not a snapshot frozen at the task boundary. It remains feedforward in the sense of Section 3.3, because what it reads is the curvature model, never the past-task performance it is meant to protect. The matrix is never formed explicitly. Fisher-vector products $v \mapsto Fv$ are computed matrix-free by the Gauss–Newton trick of Schraudolph [23]: one forward pass gives the logits (retained across the solve), each product then costs one Jacobian-vector and one vector-Jacobian product.

The solve. The filtered push $\delta\,(F_{\mathrm{rep}} + \delta I)^{-1} g_{\mathrm{cur}}$ of Equation (3.4) is computed each step by conjugate gradients over these Fisher-vector products (25 iterations, relative-residual tolerance 10^{-4}), warm-started from the previous step's solution, which keeps the per-step solve cheap across the run; the warm start is dropped at task boundaries, where the landscape shifts. The solver control of Section 5.2.4 reruns the strongest filter ($\delta = 0.03$, full buffer) at 60 iterations to separate properties of the gate from artefacts of an under-converged solve. On the full-buffer leg the replay gradient is exact over all 60k past samples while the Fisher stays estimated on a sampled batch, since a full-buffer Fisher-vector product per CG iteration would dominate the runtime.

Order of gate and momentum. When momentum is enabled, the gate is applied first: the filtered direction d is written in place of the gradient, and the optimiser's velocity buffer accumulates $v \leftarrow \mu v + d$ with the step $-\eta\, v$. The velocity is therefore built from already-filtered directions and is itself applied unfiltered. The alternative ordering, accumulating raw gradients and filtering the velocity, is a distinct design with potentially different behaviour under the re-inflation mechanism of Section 6.3; it is not implemented here and is discussed as future work in Section 7.5.

B Metric Definitions

This appendix gives the full definitions of every metric recorded by the evaluation pipeline of Section 5.1.2. The main text explains the measures the argument turns on; the tables below are the look-up reference for the complete set.

Stability-gap metrics. Computed from the dense per-step evaluation. Write a_j^{pre} for the pre-task accuracy on task j and $a_j(t)$ for its accuracy at step t of the new task.

Metric	Definition	Role
`gap_depth`	$\max_j \left(a_j^{\mathrm{pre}} - \min_t a_j(t)\right)$ over the full record window	Primary metric, worst single point of instability
`gap_area_end`	$\sum_j \int_0^T \max(0,\, b_j - a_j(t))\, dt$, trapezoidal over the full new-task record (to task end T), with $b_j = \min\left(a_j^{\mathrm{pre}}, \bar{a}_j^{\mathrm{end}}\right)$ and $\bar{a}_j^{\mathrm{end}}$ the trailing-window mean (last $\max(5, \lceil 0.1n \rceil)$ samples); accuracy at or above b_j contributes 0	Transient cost reported in the results: dip below the recovered level until it closes, excluding permanent forgetting and continued learning (units: accuracy · steps)
`recovery_steps`	The first step (counted from the transition) at which *every* past task is simultaneously back at $\geq 90\%$ of its pre-task baseline; undefined (reported as "—") if no sub-threshold dip occurs, or if recovery is not reached within the record	Duration of the transient, how long the gap stays open

Table B.1: Stability-gap metrics, recorded from dense per-step evaluation. `gap_depth` and `gap_area_end` are the two axes used in the main analysis, how deep the gap cuts and how long it stays open, together with `recovery_steps`.

Continual-learning metrics. Following De Lange et al. [4], computed at task boundaries and over the full record. Let $A(E_j, f_n)$ be the accuracy on the test set of task j evaluated at the model f_n after training stage n, and let N be the number of tasks. The full task-boundary accuracy matrix $R[i, j] = A(E_j, f_{T_i})$ is logged for every run, so any downstream metric can be reconstructed.

Metric	Definition	Role
ACC	$\frac{1}{N}\sum_j A(E_j, f_{T_{N-1}})$	Average task accuracy at the final model
FORG	$\frac{1}{N-1}\sum_{i<N-1}\left[A(E_i, f_{T_i}) - A(E_i, f_{T_{N-1}})\right]$	Average drop from right-after-learning to final
min-ACC	$\frac{1}{N-1}\sum_{i<N-1}\min\{A(E_i, f_n) : n > T_i\}$	Worst-case retention since learning
WF_w	Mean over tasks of the largest accuracy drop in any window of w consecutive evaluations ($w \in \{10, 100\}$)	Worst-case stability over short / medium horizons
WP_w	Symmetric counterpart of WF_w: the largest accuracy gain ($w \in \{10, 100\}$)	Plasticity / learning velocity
WC-ACC	$\frac{1}{N}A(E_{N-1}, f_{T_{N-1}}) + \left(1 - \frac{1}{N}\right)$ min-ACC	Worst-case trade-off, a lower bound on ACC

Table B.2: Continual-learning metrics of De Lange et al. [4], recorded at task boundaries and over the full record.

Gradient-fidelity diagnostics. Recorded at the dense cadence for the decomposition conditions only. The tracker compares the replay-buffer gradient g_{replay} against the true past-task gradient g_{true}, computed at every step by a separate forward and backward pass over the entire past-task training set (no sub-sampling, so g_{true} is the deterministic empirical past-task gradient at the current parameters).

Diagnostic	Logged name	Role
$\cos(g_{\text{replay}}, g_{\text{true}})$	`true_grad_cosine`	Buffer-side directional fidelity, the estimator-variance driver of Section 5.2.1
$\|g_{\text{replay}}\|/\|g_{\text{true}}\|$	`true_grad_mag_ratio`	Buffer-side magnitude fidelity
$\|g_{\text{new}}\|/\|g_{\text{replay}}\|$	`grad_ratio`	Update-side magnitude asymmetry, large at step 1 when $\|g_{\text{replay}}\| \approx 0$

Table B.3: Gradient-fidelity diagnostics. Each is logged as a per-step series together with its mean (and minimum, for the cosine). The first two carry the `true_grad` prefix because they are measured against g_{true}; `grad_ratio` compares g_{replay} to the current-task gradient g_{new} and its inverse drives the adaptive curriculum of Section 5.2.2.

C Complete Results

This appendix gives the full metric set for every condition of the main experimental blocks, as recorded by the aggregation pipeline of Section 5.1.2. Each row is the mean over five seeds (three for the CIFAR generalisation block) with the seed standard deviation. Accuracy-type metrics (ACC, FORG, min-ACC, WF_{100}, WP_{100}, WC-ACC) are fractions; `gap_depth` is in percentage points (pp); `gap_area_end` is the transient cost (accuracy-fraction × steps, Table B.1), the dip below the level each task recovers to, so permanent forgetting does not contribute; "Recov." is `recovery_steps`, with "—" marking conditions where every past task stayed above the 90% recovery threshold for the whole window (no recovery event to time).

C.1 rot-MNIST: The Step-Size Ladder (S-series)

The sixteen cells of the ladder of Section 5.2.1: four rungs at matched flow time $\tau = 23.5$, each on the 1k reservoir and on the 60k exact past-task gradient, at both momentum settings, all warm-started from one frozen θ_0^* per momentum setting and seed.

Condition	ACC	FORG	min-ACC	WF_{100}	WP_{100}	WC-ACC	Depth	A_τ
Momentum 0.0								
S1 η=0.1 (1k)	0.874 ± 0.008	0.020 ± 0.019	0.653 ± 0.035	0.153 ± 0.017	0.454 ± 0.020	0.770 ± 0.019	22.9 ± 4.2	0.577 ± 0.080
S1 η=0.1 (exact)	0.895 ± 0.002	-0.020 ± 0.014	0.658 ± 0.044	0.138 ± 0.026	0.453 ± 0.025	0.773 ± 0.022	22.3 ± 5.0	0.590 ± 0.172
S2 η=0.1/3 (1k)	0.876 ± 0.004	0.021 ± 0.012	0.826 ± 0.007	0.036 ± 0.004	0.365 ± 0.005	0.858 ± 0.004	5.6 ± 1.6	0.293 ± 0.044
S2 η=0.1/3 (exact)	0.897 ± 0.002	-0.021 ± 0.012	0.839 ± 0.002	0.029 ± 0.002	0.369 ± 0.006	0.865 ± 0.002	4.2 ± 1.2	0.360 ± 0.167
S3 η=0.01 (1k)	0.877 ± 0.002	0.020 ± 0.013	0.832 ± 0.006	0.031 ± 0.004	0.368 ± 0.005	0.862 ± 0.003	4.9 ± 1.4	0.275 ± 0.055
S3 η=0.01 (exact)	0.897 ± 0.002	-0.020 ± 0.012	0.842 ± 0.003	0.025 ± 0.002	0.374 ± 0.003	0.867 ± 0.002	3.9 ± 1.2	0.346 ± 0.164
S4 η=0.001 (1k)	0.877 ± 0.002	0.020 ± 0.013	0.834 ± 0.006	0.028 ± 0.004	0.369 ± 0.006	0.863 ± 0.003	4.7 ± 1.4	0.270 ± 0.052
S4 η=0.001 (exact)	0.898 ± 0.001	-0.022 ± 0.012	0.843 ± 0.003	0.023 ± 0.002	0.376 ± 0.003	0.868 ± 0.002	3.8 ± 1.2	0.341 ± 0.163
Momentum 0.9								
$S1_M$ η=0.1 (1k)	0.929 ± 0.003	0.058 ± 0.006	0.802 ± 0.023	0.087 ± 0.008	0.442 ± 0.006	0.881 ± 0.011	15.4 ± 2.2	0.105 ± 0.022
$S1_M$ η=0.1 (exact)	0.965 ± 0.001	-0.011 ± 0.004	0.814 ± 0.011	0.076 ± 0.006	0.457 ± 0.009	0.889 ± 0.006	14.1 ± 1.2	0.301 ± 0.034
$S2_M$ η=0.1/3 (1k)	0.936 ± 0.003	0.053 ± 0.003	0.855 ± 0.009	0.054 ± 0.004	0.436 ± 0.008	0.913 ± 0.004	10.1 ± 0.7	0.021 ± 0.005
$S2_M$ η=0.1/3 (exact)	0.970 ± 0.001	-0.015 ± 0.003	0.857 ± 0.005	0.053 ± 0.002	0.454 ± 0.007	0.913 ± 0.003	9.9 ± 0.5	0.122 ± 0.014
$S3_M$ η=0.01 (1k)	0.940 ± 0.002	0.048 ± 0.003	0.901 ± 0.005	0.029 ± 0.002	0.419 ± 0.006	0.937 ± 0.003	5.4 ± 0.3	0.003 ± 0.001
$S3_M$ η=0.01 (exact)	0.972 ± 0.001	-0.017 ± 0.003	0.905 ± 0.005	0.027 ± 0.001	0.435 ± 0.007	0.939 ± 0.003	5.0 ± 0.3	0.061 ± 0.012
$S4_M$ η=0.001 (1k)	0.942 ± 0.002	0.045 ± 0.002	0.910 ± 0.003	0.020 ± 0.001	0.411 ± 0.007	0.942 ± 0.002	4.6 ± 0.2	0.0006 ± 0.0001
$S4_M$ η=0.001 (exact)	0.973 ± 0.001	-0.018 ± 0.003	0.928 ± 0.003	0.014 ± 0.002	0.426 ± 0.007	0.950 ± 0.002	2.7 ± 0.4	0.040 ± 0.011

Table C.1: Step-size ladder on two-task rot-MNIST, full metrics: rungs $\eta = 0.1$, 0.1/3, 0.01 and 0.001 at 1, 3, 10 and 100 epochs, each on both buffer arms and at both momentum settings, five seeds. Depth is a maximum over accuracies and so is η-invariant. Area is *not*: `gap_area_end` is a trapezoidal integral of the drop curve against the step index, weighted by the spacing between records (units accuracy·steps, Section 5.1.2), so an identically shaped excursion reads $c = 0.1/\eta$ times larger on a rung stretched by c. The area column is therefore reported in flow time, $A_\tau = \eta\, A_{\text{steps}}$, which is the integral of the same curve against $\tau = \sum_t \eta$ and is comparable across rungs; the fixed-η blocks elsewhere in this appendix keep the step-axis value. WF_{100} **and** WP_{100} **are fixed hundred-*step* horizons and therefore span a hundredth of the flow time at $\eta = 0.001$ that they span at $\eta = 0.1$; their movement down the ladder is a units artefact of the reparametrisation (Section 6.1).**

Momentum is not simply a longer step. The grid provides two matched-effective-step diagonals, and together they say what one alone could not. At effective step 0.1, $(\eta = 0.01, \mu = 0.9)$ against $(\eta = 0.1, \mu = 0)$ separates widely, 5.0 pp / 0.061 / 0.972 against 22.3 pp / 0.590 / 0.895 on the exact arm; at effective step 0.01, $(\eta = 0.001, \mu = 0.9)$ against $(\eta = 0.01, \mu = 0)$ reads 2.7 pp / 0.040 / 0.973 against 3.9 pp / 0.346 / 0.897. The depth separation collapses from 17.3 to 1.2 pp as the matched step shrinks, which is what two legs approaching their own $\eta \to 0$ limits must do; the area separation does not, staying eightfold at the finest matched pair. What differs between the legs is therefore the shape of the residual excursion and not only its scale. This remains the secondary read Section 5.2.1 calls it: the legs warm-start from anchors trained at their own momentum and settle some seven accuracy points apart, so comparing their levels compares two operating points.

Momentum's arc is depth without tail. The within-leg contrast is both the safer comparison and the sharper one. On the sampled arm the momentum leg's flow-time area falls by more than two orders of magnitude across the ladder, $0.105 \to 0.0006$, and extrapolates to zero (-0.001), while its depth converges on 4.1 pp. In the limit the transient that survives under momentum is a dip that recovers almost at once: it keeps the instantaneous drop and loses the hanging tail. The $\mu = 0$ arc keeps both, 4.7 pp of depth and 0.27 of area that no further shortening of the step removes. Of the two-part measure of Section 5.1.2, then, momentum leaves essentially only the depth term standing.

The coarse rung reproduces the operating point in distribution. $\eta = 0.1$ on the sampled arm is the standard operating point, but it is not the archived vanilla-ER reference run (D1, Appendix C.2) and is not paired with it: the ladder skips T_0 training so that every cell can share one θ_0^*, which leaves the two protocols on different RNG streams and hence with different buffer contents seed for seed. Their distributions agree, 22.9 ± 4.2 against 19.5 ± 4.1 pp of depth and 5.77 ± 0.80 against 6.27 ± 0.47 of step-axis area, one-standard-deviation intervals overlapping on both. That is a check that warm-starting has not moved the operating point, and nothing stronger than that.

C.2 rot-MNIST: λ-Curriculum (C-series)

Vanilla ER at the standard protocol (D1: 1k reservoir, $\eta = 0.1$, one online epoch per task, the two gradients summed as they come) is the reference every rot-MNIST comparison is quoted against, and its full metric row heads each momentum block below rather than standing in a table of its own. These are the runs from which the buffer-fidelity diagnostics of Table C.4 (Appendix C.3) are taken; the step-size ladder re-runs its own $\eta = 0.1$ rung rather than reusing the row, because every cell of the ladder has to share one frozen θ_0^* (Appendix A.4).

Condition	ACC	FORG	min-ACC	WF_{100}	WP_{100}	WC-ACC	Depth	Area end	Recov.
Momentum 0.0									
Vanilla ER (baseline)	0.873 ± 0.005	0.019 ± 0.020	0.706 ± 0.056	0.147 ± 0.029	0.736 ± 0.020	0.794 ± 0.028	19.5 ± 4.1	6.3 ± 0.5	3.6 ± 0.9
C1 linear N50	0.851 ± 0.023	0.042 ± 0.021	0.707 ± 0.019	0.148 ± 0.011	0.748 ± 0.006	0.784 ± 0.021	17.5 ± 1.7	4.9 ± 0.5	35.8 ± 6.3
C2 linear N100	0.844 ± 0.025	0.046 ± 0.023	0.719 ± 0.033	0.140 ± 0.025	0.730 ± 0.006	0.786 ± 0.019	16.2 ± 4.3	3.6 ± 0.3	64.0 ± 8.6
C3 linear N200	0.828 ± 0.025	0.054 ± 0.024	0.739 ± 0.048	0.128 ± 0.043	0.688 ± 0.004	0.784 ± 0.034	14.3 ± 4.6	1.8 ± 0.6	132.8 ± 43.4
C4 adaptive	0.833 ± 0.025	0.050 ± 0.019	0.754 ± 0.025	0.111 ± 0.016	0.703 ± 0.007	0.794 ± 0.020	12.8 ± 2.9	2.2 ± 0.5	88.8 ± 6.1
C5 adaptive lmin0.05	0.835 ± 0.025	0.048 ± 0.019	0.749 ± 0.027	0.113 ± 0.016	0.702 ± 0.006	0.793 ± 0.020	13.2 ± 3.2	2.2 ± 0.5	88.8 ± 6.1
C6 adaptive lmin0.10	0.836 ± 0.026	0.048 ± 0.021	0.743 ± 0.026	0.120 ± 0.019	0.700 ± 0.007	0.791 ± 0.019	13.9 ± 3.3	2.4 ± 0.5	88.4 ± 11.0
C7 adaptive lmin0.20	0.837 ± 0.031	0.049 ± 0.024	0.743 ± 0.030	0.119 ± 0.023	0.710 ± 0.010	0.792 ± 0.024	13.9 ± 3.4	2.7 ± 0.6	64.8 ± 8.2
Momentum 0.9									
Vanilla ER (baseline)	0.928 ± 0.003	0.058 ± 0.007	0.797 ± 0.028	0.092 ± 0.016	0.808 ± 0.012	0.878 ± 0.014	15.9 ± 2.6	1.0 ± 0.2	12.6 ± 1.7
C1 linear N50	0.932 ± 0.003	0.053 ± 0.005	0.892 ± 0.004	0.041 ± 0.004	0.830 ± 0.012	0.927 ± 0.003	6.4 ± 0.5	0.2 ± 0.1	—
C2 linear N100	0.932 ± 0.003	0.051 ± 0.008	0.892 ± 0.005	0.036 ± 0.003	0.826 ± 0.012	0.926 ± 0.002	6.4 ± 0.6	0.2 ± 0.1	—
C3 linear N200	0.925 ± 0.007	0.051 ± 0.006	0.893 ± 0.004	0.031 ± 0.004	0.819 ± 0.010	0.920 ± 0.006	6.2 ± 0.6	0.1 ± 0.0	—
C4 adaptive	0.917 ± 0.001	0.019 ± 0.004	0.931 ± 0.005	0.016 ± 0.003	0.798 ± 0.010	0.915 ± 0.001	2.5 ± 0.5	0.3 ± 0.1	—
C5 adaptive lmin0.05	0.922 ± 0.001	0.023 ± 0.004	0.930 ± 0.005	0.016 ± 0.002	0.802 ± 0.009	0.921 ± 0.001	2.6 ± 0.5	0.1 ± 0.0	—
C6 adaptive lmin0.10	0.927 ± 0.002	0.028 ± 0.005	0.926 ± 0.004	0.017 ± 0.003	0.806 ± 0.009	0.927 ± 0.002	3.0 ± 0.4	0.0 ± 0.0	—
C7 adaptive lmin0.20	0.931 ± 0.002	0.035 ± 0.005	0.919 ± 0.003	0.021 ± 0.004	0.812 ± 0.009	0.930 ± 0.001	3.7 ± 0.4	0.0 ± 0.0	—

Table C.2: rot-MNIST λ-curriculum sweep, full metrics, at both momentum settings, each block headed by the vanilla-ER reference the sweep is quoted against. All conditions applied on top of standard ER (1 k reservoir). Area is on the step axis, as in every block at fixed η; the ladder converts to flow time (Table C.1).

The ingredient sweep: adaptive signal and $\lambda_{\min}$ floor. Table C.2 settles the two design choices of the shipped configuration, and both are decided in the momentum-on leg. At momentum 0 the adaptive schedule is indistinguishable from the longest linear ramp (12.8 pp / 2.2 / 0.833 against C3's 14.3 pp / 1.8 / 0.828, seeds split on depth, $p = 0.44$), so at that setting it is preferred for having no ramp length to tune rather than for its numbers. Under momentum the same pair separates: $C4_M$ reaches 2.5 pp / 0.3 / 0.917 against $C3_M$'s 6.2 pp / 0.1 / 0.925, shallower in all five seeds by ≈3.7 pp, so the feedback signal both removes the knob and beats the best setting of the knob, at a small cost in final accuracy and in the tail. The floor trades that cost back. Raising $\lambda_{\min}$ from 0 to 0.20 ($C4_M \rightarrow C7_M$) moves depth from 2.5 to 3.7 pp, worse in all five seeds, while accuracy moves from 0.917 to 0.931 and the area from 0.3 to 0.03, better in all five seeds on both; the intermediate settings $C5_M$ and $C6_M$ walk monotonically between the two ends in depth and accuracy, so the floor is a graded Pareto knob rather than a switch. $C7_M$ is the shipped point because it clears the tuned linear ramp on all three quantities (depth in five seeds of five, accuracy in four) and returns accuracy to vanilla ER's level (3.7 pp / 0.03 / 0.931 against 15.9 pp / 1.0 / 0.928, above vanilla in four of five seeds and within the seed spread). At momentum 0 the floor is not a useful knob: it costs depth (12.8 → 13.9 pp) and returns almost nothing in accuracy (0.833 → 0.837), since early T_1 progress there is set by the boundary gradient the floor is too small to redirect.

The sweep as trajectories. Figures C.1 and C.2 are the trajectory form of Table C.2, one figure per schedule family on the same 2 × 2: rows the momentum setting, columns stability (T_0) and plasticity (T_1), vanilla ER carried into every panel as the ungated reference. Read together they show why the two knobs are decided in the momentum-on leg. At momentum 0 both families trade the same way and neither trade is good: the linear ramps buy their shallower start by postponing the descent rather than preventing it, each rung releasing the brake at its own N and dropping into the same chronic, jittery band vanilla ER occupies (Figure C.1(a)), and every rung of both families pays for it in (b), where the new-task curve is displaced bodily to the right in proportion to how much schedule it carries. That displacement is the plasticity debt, and at momentum 0 nothing repays it. The momentum row is a different picture. In (c) the schedules never dip at all: they descend gently from the pre-switch level and stay above vanilla ER for essentially the whole transition instead of dropping below it and recovering, and in (d) the new-task curves have closed most of the way back onto vanilla's, the debt repaid, with the residual gap ordered by the floor: the more the schedule is allowed to brake, the more of the debt survives acceleration, which is what makes $\lambda_{\min}$ a graded Pareto knob rather than a switch. Both stability panels of a figure span the same 0.30 of accuracy on an offset floor, so a deeper-looking dip is a deeper dip.

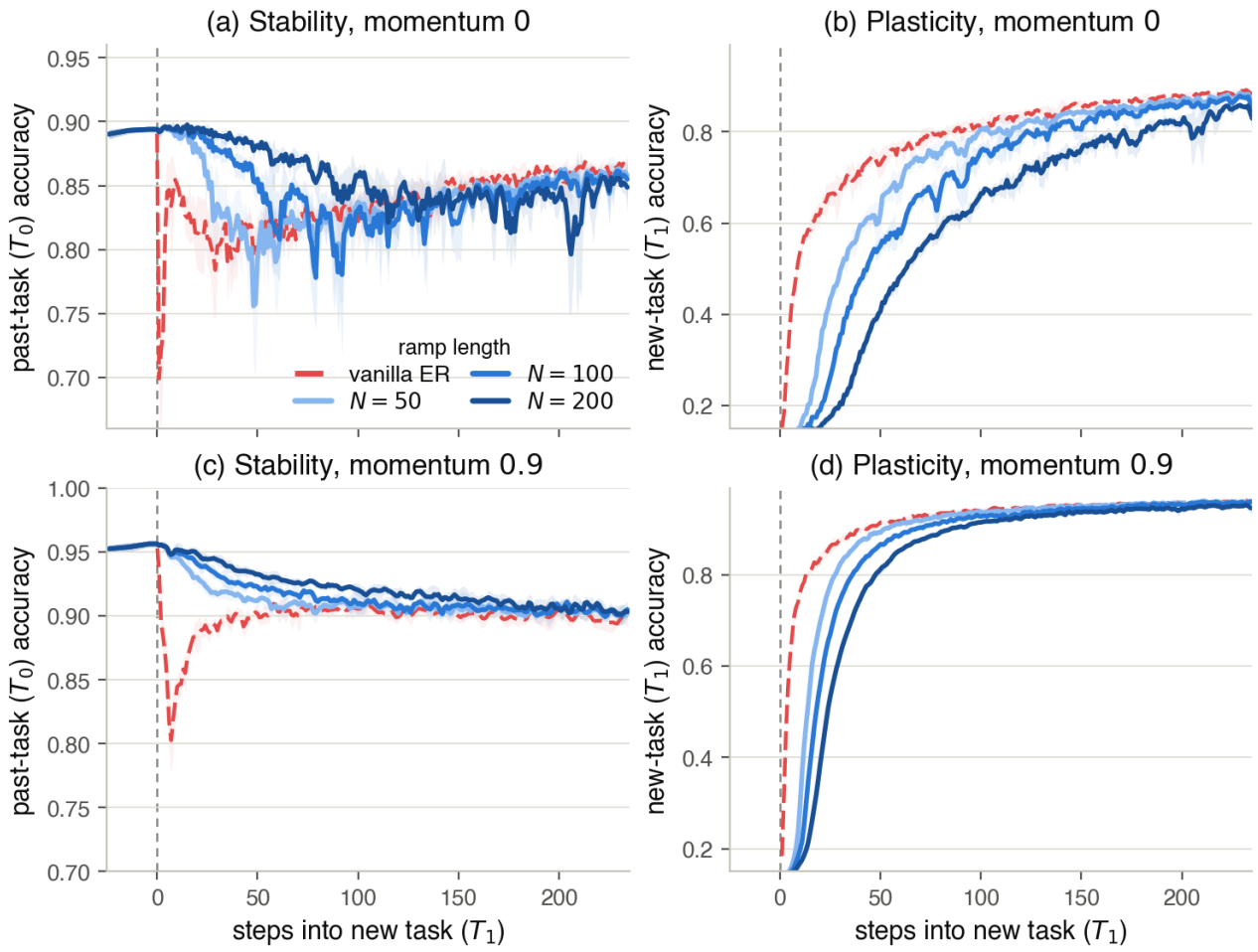


Figure C.1: The linear λ-ramps (C1–C3) across the rot-MNIST switch: per-step accuracy, seed mean with $\pm$std bands, x=0 the task switch. Rows are the momentum setting, columns the two tasks; the ramp runs short light $\rightarrow$ long dark, over vanilla ER (red, dashed). A longer ramp holds T_0 higher for longer and costs correspondingly more early T_1 progress, at momentum 0 without ever escaping the chronic band vanilla settles into.

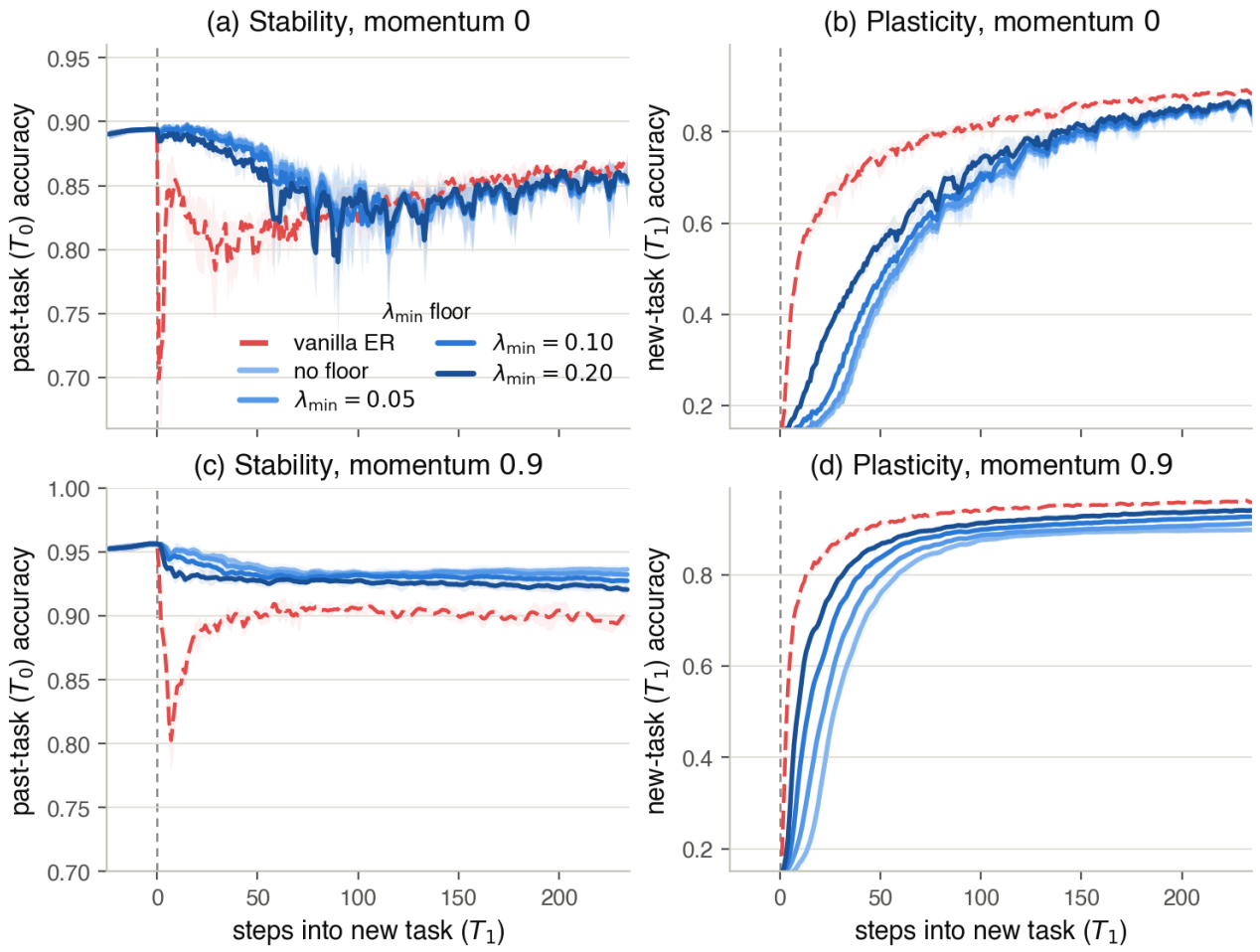


Figure C.2: The adaptive λ-curriculum and its $\lambda_{\min}$ floor (C4–C7), drawn as Figure C.1; the ramp runs unfloored light $\rightarrow$ $\lambda_{\min}$=0.20 dark. Raising the floor returns early T_1 progress (panels (b) and (d), the darker rungs leading) for a slightly deeper T_0 transient, the trade the shipped configuration C7 takes at momentum 0.9 (Table C.2).

Full-buffer momentum diagnostic (C8). Table C.3 crosses the practical curriculum with the full 60k buffer at both momentum settings, isolating momentum's two roles (Section 6.2). Source variance reduction alone (C8, full buffer, μ=0) already closes the gap to 0.2 pp, so the residual the curriculum leaves at μ=0 is estimator variance, not structure; momentum's gap-closing role is interchangeable with the full buffer. Acceleration alone (the μ=0.9 column) restores accuracy: C8$_M$ reaches 0.955, approaching the exact-gradient ceiling (the ladder's ungated $\eta = 0.1$ rung on the same 60k gradient at μ=0.9: 0.965, Table C.1), while re-introducing a small transient (2.0 pp, area 0.38). The shape and timing of this transient point to the $\lambda_{\min}$ floor meeting momentum. With the replay gradient exact and the arc removed, the only push left in the first steps is the floored fifth of the new-task gradient (the logged gradient-norm ratio confirms the schedule sits at $\lambda = 0.2$ through the dip window), and momentum amplifies a persistent push toward $1/(1-\mu) = 10$ times its per-step size: the dip accordingly opens at the boundary, bottoms out about ten steps in, and closes within about fifty as the replay gradient reasserts itself. The floor is by construction the one share of the push the source gate does not attenuate, so momentum briefly re-inflates exactly that share, a miniature of the directional gate's re-inflation. The dose-response on the same exact buffer supports the reading: the floored push without momentum costs 0.2 pp (C8), with momentum 2.0 pp (C8$_M$), and the unfloored abrupt switch with momentum 14.1 pp (the same ungated exact-gradient rung, which is not seed-paired with C8$_M$ and is read as a magnitude only). C8$_M$'s area additionally counts in full under the end-referencing convention of Section 5.1.2, since it recovers above its pre-switch level. C8/C8$_M$ are diagnostic probes: the full buffer defeats the limited-memory premise, and the practical scalar-gate configuration remains C7$_M$ (3.7 pp / 0.03 / 0.931) at 1/60th the memory.

	C7 (1k, μ=0)	C7$_M$ (1k, μ=0.9)	C8 (full, μ=0)	C8$_M$ (full, μ=0.9)
`gap_depth` (pp)	13.9	3.7	**0.2**	2.0
`gap_area_end`	2.7	0.03	**0.05**	0.38
ACC	0.837	0.931	0.818	**0.955**

Table C.3: Curriculum × full buffer × momentum on rot-MNIST, isolating momentum's two roles. Source variance reduction alone (C8) closes the gap; acceleration alone (the μ column) restores accuracy. The residual at C7 (μ=0) is variance, not structure.

The gate against the ladder's bar, rung by rung. Lowering the gap is not by itself the gate's claim, because the step size lowers it with no method at all; the claim has to be that the gate lowers it at the standard step size and in the standard number of steps (Section 6.1). Nor is there an accuracy handicap to discount from the ladder's numbers: ACC is flat to rising down both arms and min-ACC improves sharply, so what the ladder charges is compute, the epoch count scaling as $1/\eta$ at matched flow time. Read against that bar on the matched 1k arm at momentum 0, the curriculum loses on everything but the budget. C7 leaves 13.9 pp of depth at ACC 0.837 and min-ACC 0.743 in one epoch at $\eta = 0.1$; the $\eta = 0.01$ rung leaves 4.9 pp at ACC 0.877 and min-ACC 0.832 in ten. Measured against its own protocol's reference the schedule removes 5.6 pp of vanilla ER's 19.5 and spends 3.6 pp of ACC doing it (0.873 → 0.837), where the tenfold shorter step removes 18.0 pp of the ladder's distributionally matched 22.9 and gains 0.3 pp of ACC (0.874 → 0.877). The flow-time areas do land together, 0.27 for C7 against the $\eta = 0.01$ rung's 0.275 (the curriculum's step-axis 2.7 converted at $A_\tau = \eta A_{\text{steps}}$), but the two are referenced to plateaus four points apart and a lower plateau is a lower line to integrate against, so this is not a tie (Section 5.1.2).

Under momentum the comparison turns around, which is the reading Section 6.2 quotes. On the same 1k arm at momentum 0.9 the $\eta = 0.01$ rung reaches 5.4 pp at ACC 0.940 and min-ACC 0.901 after ten epochs, so one epoch of the schedule is 1.7 pp shallower and 1.8 pp better in worst-case retention than ten epochs of a tenfold shorter step, for 0.9 pp of ACC (the two protocols' $\eta = 0.1$ cells agree here as at momentum 0, 15.4 ± 2.2 against 15.9 ± 2.6 pp at ACC 0.929 against 0.928). That rung is a rung and not a limit, but the ladder's fourth rung supplies the limit as well: at $\eta = 0.001$, a hundred epochs, the same arm reaches 4.6 pp at ACC 0.942 and min-ACC 0.910, and the fine-rung extrapolation puts the $\eta \to 0$ depth of this leg at 4.1 pp. One epoch of the schedule is thus 0.9 pp shallower than a hundred epochs of a hundredfold shorter step, and its 3.7 ± 0.4 pp sits at or below what shortening the step can ultimately reach here, for 1.1 pp of ACC.

C.3 rot-MNIST: Buffer-Fidelity Diagnostics

The gradient diagnostics of Appendix B were logged on the vanilla-ER reference runs only. Each such step costs a full forward and backward pass over the whole past-task training set, which is why they are not carried elsewhere, and the stretched rungs of the ladder least of all (Appendix A.4). The rows below are therefore the directional and magnitude error that a 1,000-sample reservoir incurs against g_{true} at the standard step size, and they set the figure the ladder's exact arm is defined against: a 60k buffer replays the deterministic empirical past-task gradient, so its cosine and magnitude ratio are 1.000 by construction.

Condition	$\bar{\cos}$	$\cos_{\min}$	$\bar{\rho}$
D1 vanilla ER (μ=0)	0.711 ± 0.035	0.086 ± 0.103	1.040 ± 0.039
D1 vanilla ER (μ=0.9)	0.551 ± 0.033	0.135 ± 0.069	0.418 ± 0.014

Table C.4: Buffer-fidelity diagnostics for the vanilla-ER reference: $\bar{\cos}$ and $\cos_{\min}$ are the mean and minimum of $\cos(g_{\text{replay}}, g_{\text{true}})$ over the window; $\bar{\rho}$ is the mean of $\|g_{\text{replay}}\|/\|g_{\text{true}}\|$.

C.4 rot-MNIST: Asymmetric PER (P-series)

Table C.5 gives the full metric set for the asymmetric-PER δ sweep on both buffer legs and at both momentum settings; the δ=0.03 rows are the two the main text carries (Table 6.1), and the momentum-0.9 rows are the source of the re-inflation numbers of Section 6.3. Momentum-0.9 runs cover the three strongest filters on the standard leg and $\delta \in \{0.3, 0.1\}$ on the full-buffer leg.

Condition	ACC	FORG	min-ACC	WF_{100}	WP_{100}	WC-ACC	Depth	Area end
Standard (1k) buffer, momentum 0.0								
δ=1.0	0.869 ± 0.005	0.027 ± 0.018	0.792 ± 0.012	0.081 ± 0.009	0.733 ± 0.009	0.838 ± 0.006	9.0 ± 0.6	3.29 ± 0.65
δ=0.3	0.873 ± 0.004	0.023 ± 0.017	0.807 ± 0.012	0.068 ± 0.009	0.736 ± 0.008	0.847 ± 0.005	7.4 ± 1.1	2.32 ± 0.59
δ=0.1	0.876 ± 0.004	0.018 ± 0.016	0.822 ± 0.012	0.056 ± 0.009	0.739 ± 0.008	0.856 ± 0.005	5.9 ± 1.5	1.56 ± 0.65
δ=0.03	0.880 ± 0.005	0.012 ± 0.015	0.834 ± 0.012	0.048 ± 0.005	0.741 ± 0.007	0.862 ± 0.005	4.8 ± 1.9	1.04 ± 0.58
Standard (1k) buffer, momentum 0.9								
δ=0.3	0.929 ± 0.002	0.061 ± 0.004	0.851 ± 0.008	0.065 ± 0.004	0.813 ± 0.009	0.907 ± 0.005	10.4 ± 0.5	0.50 ± 0.17
δ=0.1	0.928 ± 0.002	0.061 ± 0.006	0.861 ± 0.008	0.060 ± 0.005	0.815 ± 0.009	0.911 ± 0.005	9.4 ± 0.5	0.38 ± 0.19
δ=0.03	0.929 ± 0.003	0.060 ± 0.009	0.871 ± 0.008	0.053 ± 0.007	0.816 ± 0.010	0.917 ± 0.005	8.5 ± 0.6	0.23 ± 0.12
Full (60k) buffer, momentum 0.0								
δ=1.0	0.898 ± 0.002	-0.025 ± 0.012	0.841 ± 0.011	0.037 ± 0.011	0.734 ± 0.006	0.865 ± 0.006	4.0 ± 1.0	2.32 ± 1.38
δ=0.3	0.900 ± 0.002	-0.029 ± 0.012	0.867 ± 0.002	0.026 ± 0.007	0.739 ± 0.005	0.878 ± 0.002	1.5 ± 1.1	1.01 ± 0.91
δ=0.1	0.901 ± 0.002	-0.032 ± 0.013	0.860 ± 0.026	0.040 ± 0.028	0.741 ± 0.004	0.875 ± 0.013	2.2 ± 1.6	0.44 ± 0.38
δ=0.03	0.903 ± 0.002	-0.035 ± 0.012	0.787 ± 0.094	0.118 ± 0.102	0.742 ± 0.004	0.838 ± 0.048	9.5 ± 8.2	0.43 ± 0.25
δ=0.03 (CG 60)	0.903 ± 0.002	-0.035 ± 0.012	0.766 ± 0.113	0.126 ± 0.109	0.742 ± 0.004	0.828 ± 0.057	11.5 ± 10.1	0.47 ± 0.28
Full (60k) buffer, momentum 0.9								
δ=0.3	0.964 ± 0.001	-0.012 ± 0.002	0.864 ± 0.009	0.052 ± 0.005	0.813 ± 0.008	0.913 ± 0.005	9.1 ± 0.8	1.91 ± 0.26
δ=0.1	0.965 ± 0.001	-0.012 ± 0.003	0.880 ± 0.008	0.044 ± 0.004	0.814 ± 0.008	0.921 ± 0.005	7.6 ± 0.7	1.57 ± 0.25

Table C.5: rot-MNIST asymmetric-PER δ sweep, full metrics: both buffer legs at both momentum settings, plus the solver control (CG 60). The momentum-0.9 rows show the re-inflation of Section 6.3: depth climbs on every leg relative to its momentum-0 counterpart, while the full-buffer accuracies ($\approx$0.965) are diagnostic values only, the full buffer defeating the limited-memory premise.

The sweep, rung by rung, on both tasks. Figure C.3 is the trajectory form of Table C.5: the whole standard-buffer sweep, both tasks, both momentum settings. Three things are visible there that the scalar summaries only assert. First, the ordering in δ in (a) is an ordering of whole trajectories and not only of their worst points: the rungs stack without crossing for the entire transition, so a stronger filter is uniformly better on the past task rather than better at the spike and worse afterwards. Second, (b) is the evidence behind the no-plasticity-cost claim of Section 6.3: the strongest filter is visibly slower over roughly the first fifty steps, the selective slowdown along the directions the sampled Fisher rates as sharp, and thereafter it is indistinguishable from vanilla ER, so the horizon-100 metrics record no cost. Third, the momentum row keeps that ordering in (c) but reads it against a baseline that has itself changed: vanilla ER's transient is shorter and the whole ladder sits 3–4 pp deeper, while (d) shows the new-task curves collapsing onto one another, momentum having removed what little separated them. Both stability panels span the same 0.30 of accuracy on an offset floor, so a deeper-looking dip is a deeper dip; the pre-switch level is higher in (c) only because momentum converges T_0 further before the switch. No δ=1.0 cell was run at momentum 0.9.

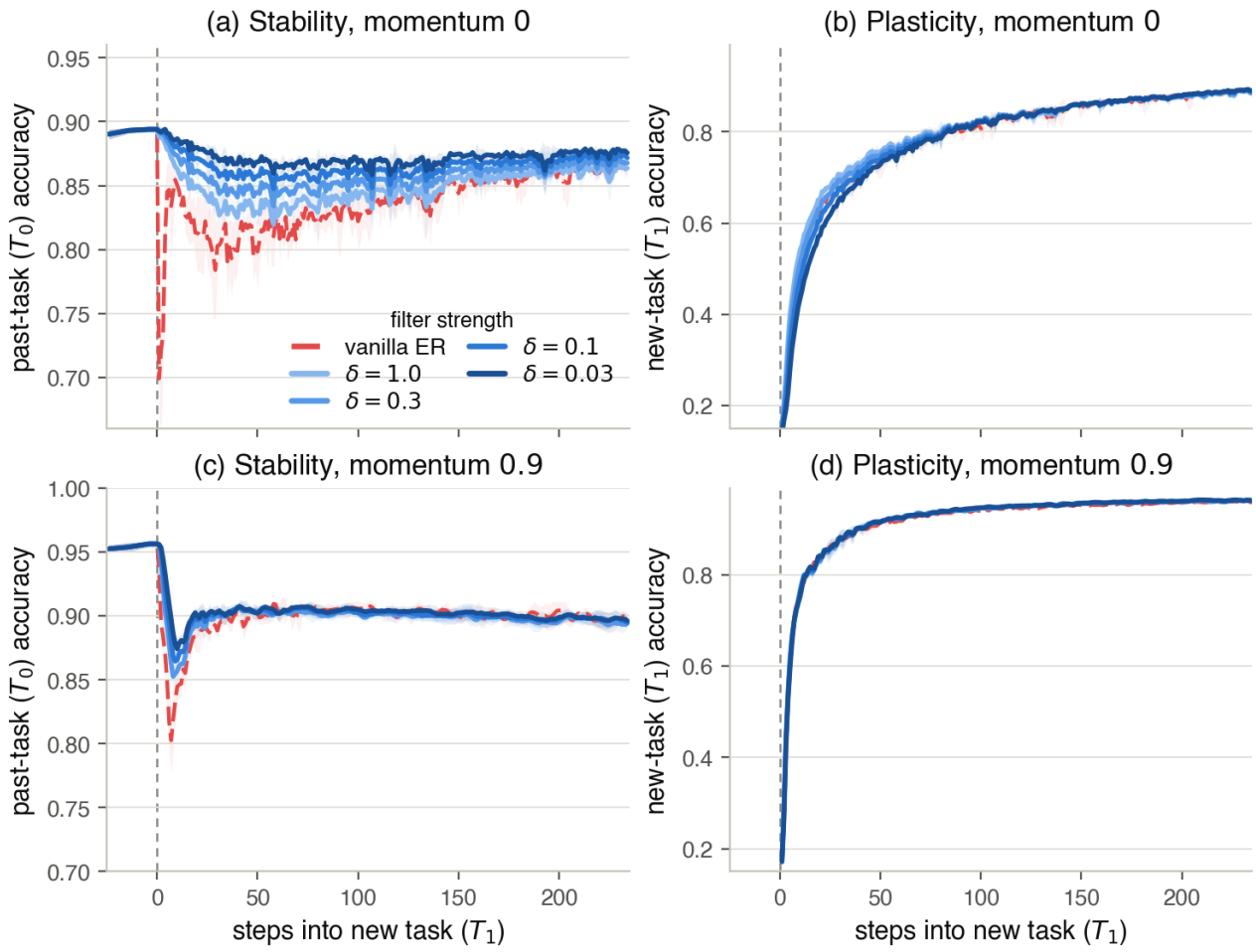


Figure C.3: The asymmetric-PER δ sweep on the standard (1k) buffer: per-step accuracy across the rot-MNIST switch, seed mean with ±std bands, x=0 the task switch. Rows are the momentum setting, columns the two tasks; the damping runs weaker filter light → stronger filter dark, over vanilla ER (red, dashed) as the ungated reference in every panel. (a) the dip shrinks monotonically as $\delta \to 0$; (b) the strongest filter lags for ∼50 steps and then tracks vanilla; (c) the same ordering, displaced 3–4 pp deeper by the accumulator; (d) under momentum the new-task curves are indistinguishable. Scalar metrics for every rung are in Table C.5.

The exact-gradient diagnostic and its cliffs. The full (60k) buffer is a diagnostic probe rather than an operating point: replaying the exact past-task gradient switches the variance driver off, so whatever excursion survives is the arc together with the standard step's overshoot of it (Section 6.1). Read that way the leg behaves as the account predicts down to $\delta = 0.1$, the excursion shrinking with the filter. The one place the sweep breaks monotonicity is the strongest setting, $\delta = 0.03$, where mean depth jumps to 9.5 pp with a std of 8.2: in 3 of 5 seeds the past-task curve holds flat for roughly two hundred steps and then slides off a late cliff, the red per-seed traces in Figure C.4. A solver control rules out the most straightforward explanation. The filter solves for its preconditioned direction by conjugate gradient, so an under-converged solve at the strongest damping would be the obvious suspect; rerunning the setting with the CG solver at 60 iterations instead of the default 25 (row δ=0.03 *(CG 60)* of Table C.5) reproduces the cliffs at the same seeds and the same steps, and if anything deepens them (11.5 ± 10.1 pp mean depth against 9.5 ± 8.2). They are not a numerical artefact of the solve. The cliffs came as a surprise: the account made no prediction of a late, abrupt failure, and nothing in the rest of the sweep hints at one. They are also confined to the exact-gradient diagnostic: at the same δ on the standard 1k buffer no seed shows one (depth 4.8 ± 1.9 pp, the monotone end of the sweep). The failure is tied to the exact replay gradient: the sampling jitter of a practical buffer appears to protect against it, the same noise that elsewhere drives the overshoot here keeping the iterate from settling into the escape direction. Why the cliffs happen we can only partly say. A reading consistent with the data is a blind spot of the gate's model of the past task: the filter passes new-task pressure through directions its sampled Fisher rates as flat, and in these seeds that trust appears to be misplaced further out along the trajectory. We offer this as a hypothesis, not an

established mechanism. What the control does establish is that the strongest filter can fail late and abruptly when nothing perturbs it. Because the failure is confined to the diagnostic leg, it is unlikely in a practical configuration, but it exposes a real weakness of feedforward control.

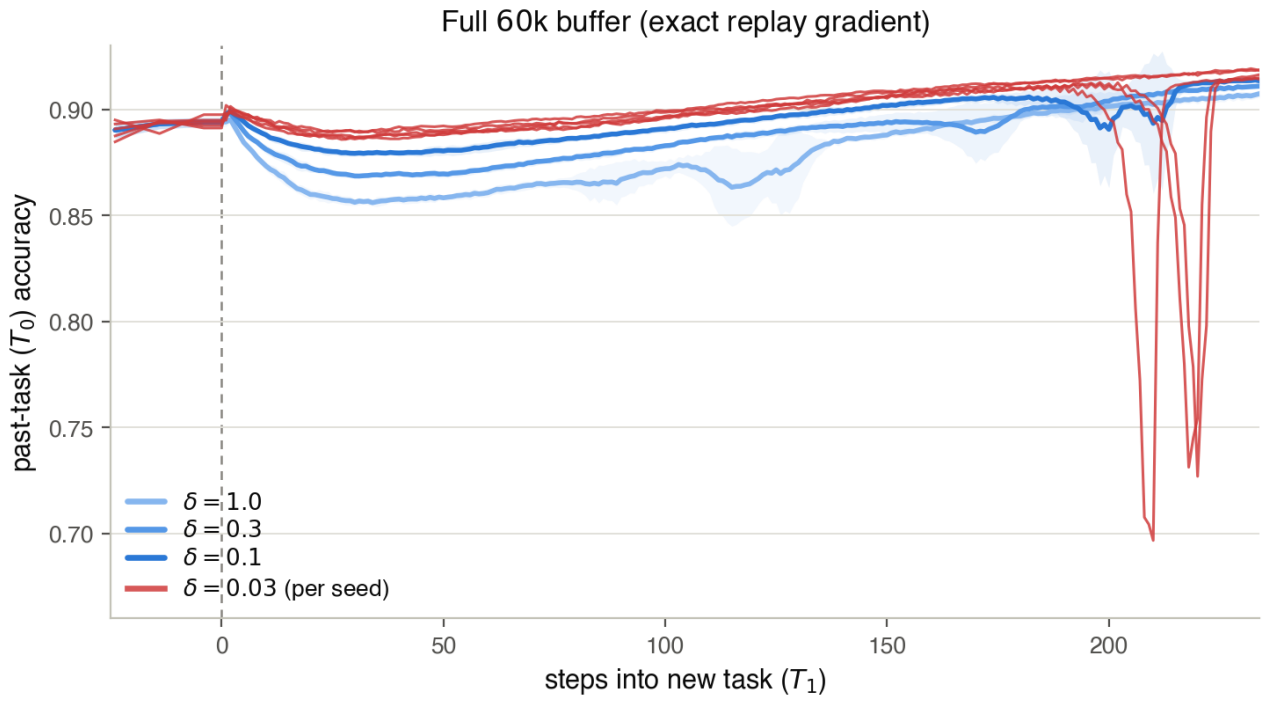


Figure C.4: The exact-gradient diagnostic leg of the δ sweep: per-step past-task (T_0) accuracy across the rot-MNIST switch at momentum 0 on the full (60k) buffer, where the variance driver is off and the residual excursion is the arc together with the standard step's overshoot of it. The excursion shrinks with the filter for $\delta \geq 0.1$ (seed mean with ±std bands, weaker filter light → stronger filter dark), but at the strongest setting δ=0.03 the gate fails late and abruptly: shown per seed (critical red), 3 of 5 seeds hold flat for ∼200 steps and then slide off a cliff. The failure is confined to this diagnostic leg; at the same δ on the standard 1k buffer no seed shows one (Figure C.3(a), depth 4.8 ± 1.9 pp).

C.5 rot-MNIST: Symmetric vs. Asymmetric Preconditioning

Leaving the restoring gradient g_{rep} at full strength is the directional gate's defining choice (Section 6.3). The alternative is to precondition the summed update, $d = \delta(F_{\text{rep}} + \delta I)^{-1}(g_{\text{cur}} + g_{\text{rep}})$, the symmetric variant that passes both gradients through the same metric. Because g_{rep} is the gradient of the replay loss, it lies in the top eigenspace of that loss's own Fisher F_{rep}, so the symmetric filter shrinks the restoring gradient *harder* than the harmful push it is meant to brake and cannot preferentially protect the past task. Table C.6 sweeps the same δ on both variants, and they are mirror images. As the filter strengthens ($\delta \to 0$) the asymmetric gate's forgetting falls (FORG 0.027 → 0.012) and its accuracy rises, while the symmetric gate's forgetting *climbs* (0.030 → 0.066) and its accuracy falls; every one of these four trends holds across all five seeds. Symmetric depth barely moves across the sweep (5.7 → 6.7 pp) and at δ=1.0 even sits below the asymmetric gate's (5.7 vs. 9.0 pp), because filtering the whole update also damps the per-step *expression* of the dip. This is the area caveat of Section 5.1.2 made concrete: the symmetric gate's end-referenced area collapses toward zero (2.7 → 0.01) not because the dip recovers but because it stops recovering, settling into a permanent accommodation that the area excludes and the forgetting metric records. Read on depth or transient area the symmetric gate looks competitive; read on forgetting and final accuracy it degrades monotonically as its filter strengthens. Asymmetry is what turns the filter from a reshuffling of the damage into a net removal of it.

	Asymmetric (g_{rep} free)				Symmetric (joint)			
δ	depth (pp)	area	FORG	ACC	depth (pp)	area	FORG	ACC
1.0	9.0	3.29	0.027	0.869	5.7	2.69	0.030	0.872
0.3	7.4	2.32	0.023	0.873	5.9	1.65	0.042	0.867
0.1	5.9	1.56	0.018	0.876	6.2	0.34	0.057	0.861
0.03	4.8	1.04	0.012	0.880	6.7	0.01	0.066	0.857

Table C.6: Asymmetric versus symmetric preconditioning on the standard 1k buffer at momentum 0, five seeds, sweeping the shared damping δ. Both variants filter through the replay Fisher F_{rep}; they differ only in whether the metric is applied to the new-task push alone (asymmetric, g_{rep} left free) or to the summed update (symmetric). As $\delta \to 0$ the asymmetric gate improves on every axis while the symmetric gate degrades on every axis; symmetric depth stays flat because the filter suppresses the per-step expression of the dip even as the underlying forgetting grows, and the symmetric end-referenced area collapses toward zero for the same reason: the past task stops recovering, so the loss registers in FORG rather than in the transient area (Section 5.1.2).

C.6 CIFAR-10: Headline Block

Condition	ACC	FORG	min-ACC	WF_{100}	WP_{100}	WC-ACC	Depth	Area end
G1 vanilla BN	0.723 ± 0.008	0.010 ± 0.019	0.644 ± 0.023	0.127 ± 0.015	0.484 ± 0.020	0.667 ± 0.016	11.8 ± 2.5	17.5 ± 4.6
G2 adaptive BN	0.722 ± 0.011	0.006 ± 0.016	0.713 ± 0.017	0.070 ± 0.010	0.469 ± 0.016	0.698 ± 0.013	4.8 ± 0.7	3.6 ± 2.2
G3 asym. PER BN	0.715 ± 0.011	-0.008 ± 0.018	0.698 ± 0.023	0.124 ± 0.022	0.460 ± 0.025	0.682 ± 0.023	5.9 ± 4.3	3.3 ± 3.0
G4 vanilla GN	0.730 ± 0.011	-0.023 ± 0.030	0.309 ± 0.111	0.294 ± 0.097	0.415 ± 0.058	0.507 ± 0.054	41.2 ± 13.5	34.9 ± 23.7
G5 adaptive GN	0.731 ± 0.020	-0.026 ± 0.026	0.670 ± 0.055	0.095 ± 0.033	0.340 ± 0.052	0.687 ± 0.037	5.2 ± 2.1	1.7 ± 0.9
G6 asym. PER GN	0.741 ± 0.009	-0.035 ± 0.018	0.673 ± 0.023	0.080 ± 0.009	0.330 ± 0.024	0.696 ± 0.015	5.6 ± 1.1	4.8 ± 3.8

Table C.7: CIFAR-10 headline block, full metrics (ResNet-18, momentum 0.9, buffer 5,000, five seeds). "adaptive" is the practical curriculum with $\lambda_{\min} = 0.20$; "asym. PER" runs at $\delta = 0.1$.

The gates' residual batch-norm dip. The small gap the gates keep under batch norm is a genuine boundary transient: in Figure 6.4(a) both gated curves dip together over the first steps after the switch, by roughly the tabulated 5 pp, and recover to their plateau within a few tens of steps. A reading consistent with this pattern is the normalisation pathway of Section 5.2.5. BatchNorm's running statistics update on every forward pass, whatever the gate does to the gradient, so the first corrupted batches shift them away from the clean task and no gate on the update can hold that back; the clean replay samples in every batch pull the statistics back, which matches the fast recovery. The group-norm contrast supports the reading: with no cross-batch statistics to disturb, the curriculum holds T_0 flat through the same boundary (Figure 6.4(b)) and its transient area falls to roughly half its batch-norm value (1.7 vs. 3.6). This is a reading rather than an established mechanism; a control that freezes the running statistics across the switch would isolate it.

C.7 CIFAR-10: Generalisation Block

Each shift pairs the practical curriculum ("curric.": adaptive $\lambda_{\min} = 0.20$, buffer 5,000) against a vanilla-ER control on the same shift, at the indicated normalisation, over three seeds.

Condition	ACC	FORG	min-ACC	WC-ACC	Depth	Area end
BatchNorm						
shot noise (vanilla)	0.724 ± 0.011	0.017 ± 0.010	0.633 ± 0.017	0.663 ± 0.014	13.1 ± 3.0	21.2 ± 3.9
shot noise (curric.)	0.723 ± 0.014	0.010 ± 0.010	0.708 ± 0.032	0.697 ± 0.016	5.6 ± 1.0	5.8 ± 1.5
contrast (vanilla)	0.671 ± 0.026	-0.008 ± 0.006	0.610 ± 0.063	0.586 ± 0.048	15.4 ± 3.9	33.4 ± 4.5
contrast (curric.)	0.671 ± 0.044	-0.012 ± 0.007	0.727 ± 0.034	0.643 ± 0.048	3.7 ± 1.1	2.2 ± 0.9
3-task (vanilla)	0.740 ± 0.006	-0.011 ± 0.013	0.670 ± 0.015	0.691 ± 0.010	3.3 ± 0.4	6.0 ± 0.7
3-task (curric.)	0.745 ± 0.010	-0.021 ± 0.008	0.677 ± 0.025	0.696 ± 0.017	6.9 ± 2.7	3.9 ± 0.1
GroupNorm						
shot noise (vanilla)	0.718 ± 0.021	-0.036 ± 0.036	0.244 ± 0.075	0.469 ± 0.032	46.3 ± 12.0	54.4 ± 40.8
shot noise (curric.)	0.705 ± 0.061	-0.028 ± 0.012	0.638 ± 0.108	0.656 ± 0.087	7.3 ± 5.9	2.4 ± 1.1
contrast (vanilla)	0.777 ± 0.022	-0.067 ± 0.021	0.641 ± 0.061	0.710 ± 0.040	8.4 ± 4.4	3.3 ± 0.6
contrast (curric.)	0.778 ± 0.032	-0.070 ± 0.013	0.668 ± 0.077	0.724 ± 0.054	3.9 ± 3.1	1.2 ± 0.8
3-task (vanilla)	0.722 ± 0.020	-0.023 ± 0.015	0.468 ± 0.043	0.550 ± 0.023	4.4 ± 0.4	10.3 ± 3.6
3-task (curric.)	0.721 ± 0.019	-0.025 ± 0.020	0.421 ± 0.114	0.517 ± 0.070	32.8 ± 14.5	32.0 ± 19.5

Table C.8: CIFAR-10 generalisation block, full metrics. The two-task novel corruptions (shot noise, contrast) favour the curriculum under both normalisations; the three-task group-norm sequence is the outlier analysed below.

The three-task group-norm failure. The three-task sequence is the one place the curriculum underperforms its control (depth 32.8±14.5 pp against vanilla's 4.4±0.4, Table C.8; Figure C.5), and the logged gradient-norm traces locate the failure in the conditioning of the feedback signal. The adaptive ratio $r = \|g_{\text{replay}}\|/\|g_{\text{new}}\|$ carries information through the boundary ordering of Section 5.2.2: at a well-separated switch the new-task gradient dwarfs the replay gradient, so r starts near zero and its rise tracks the replay gradient reasserting itself. The first boundary of this sequence behaves that way (clean → Gaussian: first-step new-task loss 1.9–2.1, $r \approx 0.2$), and the schedule holds every past task flat through it. The second boundary never establishes the ordering: Gaussian and shot noise are near-identical, so the incoming model already fits the new task (T_2 accuracy 0.67–0.71 before its first step, new-task loss 0.5–0.7), leaving $\|g_{\text{new}}\|$ small, while $\|g_{\text{replay}}\|$ is small because the model is converged on the replayed tasks. The ratio becomes a quotient of two near-zero norms: under group norm it reads 0.5–1.3 from the first step and fluctuates step to step, so the EMA drives λ from the floor to ≈0.7–0.8 within fifty steps and back to $\lambda_{\min}$ within a few hundred. The gate is moved by noise, not by damage.

What follows, in all three seeds, is not a stability gap but an optimisation instability. Within the first forty steps the replay loss itself diverges (0.2–0.4 → 0.8–2.1) and every task falls together, *including the one being trained* (T_2: 0.71 → 0.29, 0.69 → 0.30, 0.67 → 0.51 across seeds), before the run recovers; the per-seed depth scales with the divergence, which is where the ±14.5 pp spread comes from. Because all tasks crash together, the depth and area of the three-task group-norm rows record this instability, not the past-task transient the rest of the study measures. Two controls isolate the trigger. The vanilla run crosses the same boundary with a flat replay loss and 4.4 pp of depth, so the near-identical boundary is harmless on its own; the batch-norm run executes the identical schedule through the same boundary, with the same ill-conditioned early ratio, without instability (Figure C.5), so the noise-driven λ is not sufficient either. The failure needs both the unconditioned signal and the group-norm backbone's boundary sensitivity, consistent with the far deeper group-norm transients vanilla ER shows on the same corruption family (shot noise: 46.3 vs 13.1 pp, Table C.8). We therefore read the episode as a reliability boundary of the gradient-norm-ratio signal rather than of the gate principle: the ratio loses its meaning when both norms become small, and the design admits direct guards: freezing λ when both norms fall below a reference scale, or reading a damage signal that does not vanish at a quiet boundary, such as the replay-loss excess over its pre-switch level.

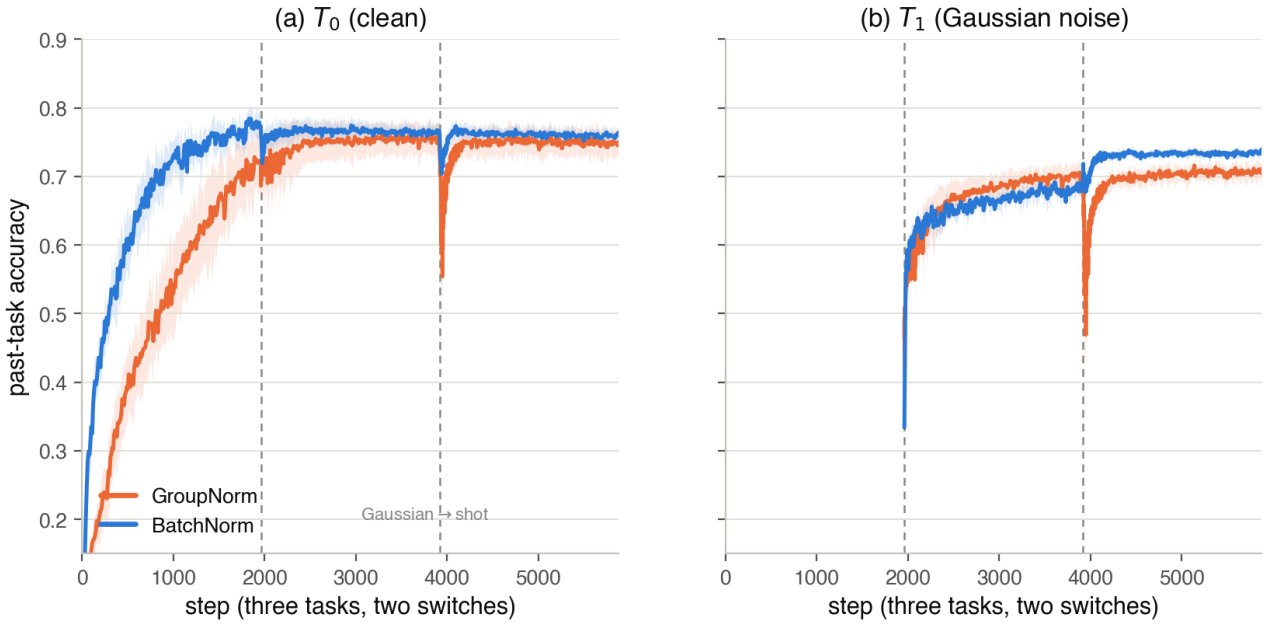


Figure C.5: Per-step past-task accuracy across the three-task CIFAR-10 generalisation sequence (none → Gaussian → shot, ResNet-18, practical curriculum, three seeds with ±std bands), comparing the group-norm run (orange) against its batch-norm counterpart (blue). The two switches (dashed) fall at ∼2k and ∼4k steps. The first boundary (none → Gaussian) barely perturbs the past tasks in either norm. The second (Gaussian → shot, two near-identical corruptions) drives a deep transient in T_0 (a) and T_1 (b) under group norm only, while the batch-norm run holds flat through both switches. The dip is thus localised entirely to the second transition and to group norm, the adaptive-ratio reliability boundary analysed above.

C.8 CIFAR-10: Momentum Cross and Compute Cost

This subsection collects the three deep-backbone checks summarised in Section 6.5: the momentum cross (Table C.9), the generalisation block (Table C.8 above), and the compute cost (Table C.10).

Momentum cross. The cross repeats the composability test of Section 5.2.3 on the ResNet, pairing momentum-0 runs at group norm against the momentum-0.9 rows, and it turns out to be only weakly informative. At momentum 0 the ResNet under-converges the past task in ten epochs (vanilla min-ACC 0.11, ACC 0.65), so every momentum-0 depth sits on a degraded baseline. On that baseline both gates improve with momentum by similar margins (curriculum 28.6 → 5.2 pp, asymmetric PER 22.6 → 5.6 pp), so the cross cannot separate the predicted source-attenuation composition from a general convergence gain, and the prediction that momentum re-inflates the directional gate does not appear; the vanilla null (40.5 → 41.2 pp) is directionally consistent with the account but rests on the same degraded baseline. We therefore draw no mechanism conclusion from this cross in either direction: the momentum signatures rest on the rot-MNIST results, where the MLP isolates them cleanly (Sections 6.2 and 6.3), and a clean deep-backbone test would need a converged momentum-0 baseline (Section 7.4).

Method (GroupNorm)	depth, μ=0	depth, μ=0.9
Vanilla ER	40.5 ± 4.0	41.2 ± 13.5
Curriculum ($\lambda_{\min}$=0.20)	28.6 ± 8.8	5.2 ± 2.1
Asym. PER (δ=0.1)	22.6 ± 9.7	5.6 ± 1.1

Table C.9: CIFAR-10 momentum cross (group norm, buffer 5,000, five seeds), gap depth in pp. All momentum-0 cells sit on an under-converged baseline (vanilla min-ACC 0.11), so the cross is read as uninformative for the momentum signatures rather than as partial confirmation (see text).

Compute cost. Cost is measured with a dedicated benchmark: one full two-task run per gate in its headline configuration (curriculum: adaptive schedule, $\lambda_{\min}$=0.20; asymmetric PER: δ=0.1, 25 warm-started CG iterations; momentum 0.9, group norm and buffer 5,000 on CIFAR), five seeds each on a single NVIDIA RTX PRO 6000, with per-step stability evaluation disabled so the numbers reflect method compute alone. Wall-clock and CPU time come from `getrusage`; GPU utilisation, power, and memory are sampled at 1 Hz with `nvidia-smi`, and energy is the integrated power. On the ResNet-18 the split is stark (Table C.10). The scalar gate adds one gradient-norm ratio and an EMA update to a standard ER step, and its run is not even compute-bound: the GPU sits half-idle at 49% utilisation and 259 W. The directional gate's 25 conjugate-gradient Fisher-vector products per step, each on the order of a backward pass, saturate the same card (96%, 367 W) for 16× as long, and since energy is power times time the two factors compound to a 24× energy gap (13 vs. 320 Wh per run). Memory is not the price: peak GPU memory differs by 11%. On the rot-MNIST MLP both gates finish in under half a minute at near-idle utilisation (the wall-clock is fixed startup, not method compute), so no meaningful ratio exists there and the cost question is decided entirely on the deep backbone. The ratios are the transferable claim; the absolute numbers are specific to this card, and because ∼15–30 s of each run is startup, the training-only ratio is if anything slightly above 16×.

Dataset	Gate	wall [s]	util [%]	power [W]	energy [Wh]	mem [MB]
CIFAR-10	Curriculum (scalar)	198 ± 7	49	259	13	4,203
(ResNet-18)	Asym. PER (δ=0.1)	$3{,}157 \pm 11$	96	367	320	4,679
rot-MNIST	Curriculum (scalar)	25 ± 1	<1	—	0.4	749
(MLP)	Asym. PER (δ=0.1)	29 ± 1	8	—	0.5	769

Table C.10: Compute cost of one full two-task run (mean over five seeds, ± sd on wall-clock; single RTX PRO 6000; per-step evaluation disabled): wall-clock, mean GPU utilisation and power, integrated GPU energy, and peak GPU memory. On the ResNet-18 the directional gate costs ∼16× the wall-clock and ∼24× the energy of the scalar gate at similar peak memory. The MLP rows are dominated by fixed startup on a near-idle GPU, so their power draw is idle draw (omitted) and no ratio is meaningful.

C.9 Long-Sequence rot-MNIST (L-series)

Condition	ACC	FORG	min-ACC	WF_{100}	WP_{100}	WC-ACC	Depth	Area end
Momentum 0.0								
L1 vanilla ER	0.861 ± 0.004	0.060 ± 0.004	0.670 ± 0.055	0.207 ± 0.053	0.443 ± 0.011	0.722 ± 0.043	21.0 ± 5.8	3.14 ± 0.77
L2 curriculum $\lambda_{\min}$=0.20	0.860 ± 0.002	0.031 ± 0.005	0.828 ± 0.007	0.073 ± 0.005	0.400 ± 0.003	0.840 ± 0.005	4.1 ± 0.8	1.62 ± 0.42
L3 asym. PER δ=0.1	0.870 ± 0.003	0.053 ± 0.006	0.843 ± 0.008	0.038 ± 0.007	0.381 ± 0.003	0.862 ± 0.007	4.7 ± 1.2	0.38 ± 0.24
Momentum 0.9								
L1 vanilla ER	0.889 ± 0.007	0.096 ± 0.008	0.845 ± 0.007	0.065 ± 0.008	0.317 ± 0.006	0.870 ± 0.006	8.9 ± 1.7	1.68 ± 0.69
L2 curriculum $\lambda_{\min}$=0.20	0.897 ± 0.002	0.071 ± 0.003	0.871 ± 0.005	0.041 ± 0.004	0.364 ± 0.005	0.888 ± 0.004	5.4 ± 0.6	0.60 ± 0.14
L3 asym. PER δ=0.1	0.884 ± 0.004	0.104 ± 0.004	0.849 ± 0.006	0.054 ± 0.004	0.325 ± 0.005	0.873 ± 0.005	7.5 ± 1.5	0.80 ± 0.63

Table C.11: Five-task rot-MNIST (rotations $[0, 30, 60, 90, 120]^\circ$, four consecutive transitions, five seeds), the L-series: both gates in their practical configurations against vanilla ER, at both momentum settings. Depth is the worst single-boundary drop and area the end-referenced transient, aggregated as in the two-task blocks. At momentum 0 both gates cut the depth from 21 pp to ≈4 pp; under momentum the curriculum keeps the best cumulative retention while the re-inflated directional gate falls slightly behind vanilla on FORG (0.104 vs. 0.096, all five seeds).

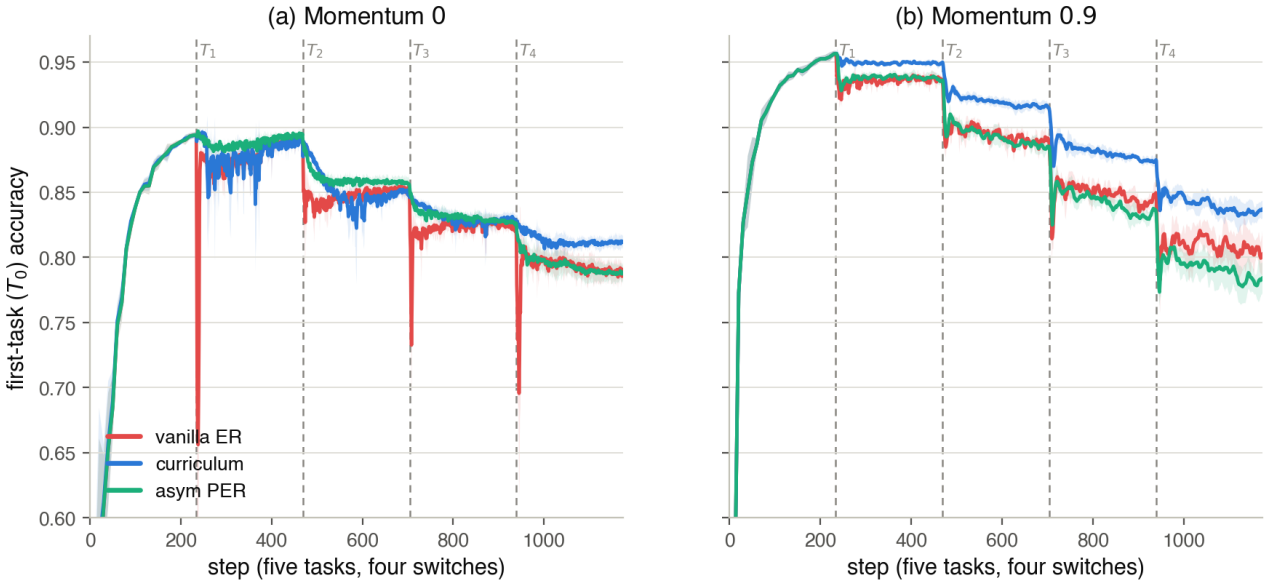


Figure C.6: Five-task rot-MNIST (rotations $[0, 30, 60, 90, 120]^\circ$, four consecutive switches, five seeds with ±std bands), the L-series: first-task T_0 accuracy across the whole sequence for vanilla ER (red), the practical curriculum $\lambda_{\min}$=0.20 (blue) and asymmetric PER δ=0.1 (aqua); dashed lines mark the four switches, and the two panels are the two momentum settings. (a) Momentum 0: both gates hold T_0 almost flat through every boundary, asymmetric PER the smoothest, while vanilla ER drops sharply at each switch. (b) Momentum 0.9: PER's boundary transients return as momentum re-inflates its per-step filter (Section 6.3); the curriculum keeps the boundaries smoothest and holds the highest plateau (Table C.11).

D Derivation of the Trajectory Bound

This appendix derives the two results used in Section 3.1: the replay loss is monotone along the reference path (Proposition 1), and a gradient-flow iterate under a linear ramp tracks that path with $O(1/N)$ lag (Proposition 2), which together give the transient bound of Equation (3.2).

Setup. For $\lambda \in [0, 1]$ define the curriculum objective

$$L_\lambda(\theta) \;=\; L_{\text{replay}}(\theta) + \lambda\, L_{\text{current}}(\theta), \qquad H_\lambda(\theta) \;=\; H_{\text{replay}}(\theta) + \lambda\, H_{\text{current}}(\theta), \tag{D.1}$$

with $H_{\text{replay}} = \nabla^2 L_{\text{replay}}$ and $H_{\text{current}} = \nabla^2 L_{\text{current}}$. We make two assumptions.

(A1) L_{replay} and L_{current} are C^2 in a neighbourhood of the path considered below.

(A2) There is a C^1 branch of critical points $\lambda \mapsto \Theta^*(\lambda)$ of L_λ with $\Theta^*(0) = \theta_0^*$ and $H_\lambda(\Theta^*(\lambda)) \succ 0$ for all $\lambda \in [0,1]$.

Assumption (A2) asks for a tracked branch of strict local minima, not global convexity. Near $\lambda = 0$ it holds automatically: θ_0^* is a non-degenerate local minimum of L_{replay}, so $H_0(\theta_0^*) \succ 0$ and the implicit function theorem yields a unique C^1 branch locally. Its persistence is controlled by Weyl's inequality, $\mu_{\min}(H_\lambda) \geq \mu_{\min}(H_{\text{replay}}) + \lambda\, \mu_{\min}(H_{\text{current}})$, with $\mu_{\min}$ the smallest eigenvalue: positive-definiteness survives until the new task's negative curvature outweighs the old task's, and holds for all λ whenever $H_{\text{current}} \succeq 0$ along the branch. On a deep non-convex backbone (A2) is a local modelling assumption, which is why Section 3.1 presents the bound as motivation rather than a guarantee.

Proposition 1 (Monotonicity of the replay loss)**.** *Under (A1)–(A2),*

$$\frac{d}{d\lambda} L_{replay}(\Theta^*(\lambda)) \;=\; \lambda \nabla L_{current}^\top H_\lambda^{-1} \nabla L_{current} \;\geq\; 0 \qquad \textit{for all } \lambda \in [0,1],$$

where $\nabla L_{current}$ and H_λ are evaluated at $\Theta^*(\lambda)$. In particular $L_{replay}(\Theta^*(\lambda)) \leq L_{replay}(\theta^*_{joint})$ for all λ, with $\theta^*_{joint} = \Theta^*(1)$.

Proof. The branch satisfies the first-order condition

$$\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0. \tag{D.2}$$

Differentiating (D.2) in λ and collecting terms gives $H_\lambda \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}} = 0$, hence

$$\frac{d\Theta^*}{d\lambda} = -\, H_\lambda^{-1} \nabla L_{\text{current}}. \tag{D.3}$$

By the chain rule and (D.2),

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) = \nabla L_{\text{replay}}^\top \frac{d\Theta^*}{d\lambda} = (-\lambda \nabla L_{\text{current}})^\top \left(-H_\lambda^{-1} \nabla L_{\text{current}}\right) = \lambda \nabla L_{\text{current}}^\top H_\lambda^{-1} \nabla L_{\text{current}},$$

which is non-negative because $\lambda \geq 0$ and $H_\lambda^{-1} \succ 0$ by (A2). Integrating from λ to 1 gives the final statement. □

Proposition 2 ($O(1/N)$ tracking lag)**.** *Let $\theta(t)$ follow the gradient flow $\dot\theta = -\nabla L_{\lambda(t)}(\theta)$ under the linear ramp $\lambda(t) = t/N$, with $\theta(0) = \theta_0^*$, and let $e(t) = \theta(t) - \Theta^*(\lambda(t))$ denote the tracking error. Suppose, in addition to (A1)–(A2), that along the branch $\mu_{\min}(H_\lambda) \geq \mu > 0$ and $\|\nabla L_{current}\| \leq g$. Then, to first order in e, the error settles at*

$$\|e\| \;\leq\; \frac{1}{N} \left\|H_\lambda^{-1}\right\| \left\|\frac{d\Theta^*}{d\lambda}\right\| \;\leq\; \frac{g}{\mu^2 N} \;=\; O\!\left(\frac{1}{N}\right).$$

Proof. Differentiating the error and inserting the flow and the chain rule for the moving target,

$$\dot e = -\nabla L_{\lambda(t)}(\theta(t)) - \frac{d\Theta^*}{d\lambda}\,\dot\lambda, \qquad \dot\lambda = \frac{1}{N}.$$

Linearising the gradient about the minimiser, $\nabla L_\lambda(\theta) = H_\lambda\, e + O(\|e\|^2)$ since $\nabla L_\lambda(\Theta^*) = 0$, yields the error dynamics

$$\dot e \;\approx\; -H_\lambda\, e \;-\; \frac{1}{N}\frac{d\Theta^*}{d\lambda}.$$

Because $H_\lambda \succeq \mu I$, these dynamics contract toward the quasi-static equilibrium $e^\dagger = -\frac{1}{N} H_\lambda^{-1} \frac{d\Theta^*}{d\lambda}$; bounding $\|H_\lambda^{-1}\| \leq 1/\mu$ and, by (D.3), $\|d\Theta^*/d\lambda\| \leq g/\mu$ gives the stated bound. The argument is a first-order, continuous-time approximation: it captures how the lag scales with the ramp length N, not an exact discrete-time constant. □

The transient bound. Expanding the replay loss about the reference point, $L_{\text{replay}}(\theta(t)) = L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top e + O(\|e\|^2)$, Proposition 1 bounds the first term by $L_{\text{replay}}(\theta^*_{\text{joint}})$ and Proposition 2 bounds the correction by $O(1/N)$ (the gradient norm $\|\nabla L_{\text{replay}}(\Theta^*)\| = \lambda \|\nabla L_{\text{current}}\| \leq g$ is bounded along the branch). Hence

$$\max_t L_{\text{replay}}(\theta(t)) \;\leq\; L_{\text{replay}}(\theta^*_{\text{joint}}) + O\!\left(\frac{1}{N}\right): \tag{D.4}$$

the permanent price is the first term, and the transient contribution of the tracking lag shrinks as the ramp lengthens, which is Equation (3.2).